# MovieMagic: A Web-Based Online Movie Ticket Booking System

## Project Description:

**MovieMagic** is a full-stack web application designed to simplify the process of browsing movies and booking tickets through a single, user-friendly platform. The system provides a smooth booking workflow where users can register/login, explore currently available movies, view details like show timings and pricing, select seats, and confirm bookings in a few steps. The application is built using **Flask** as the backend framework, with a responsive frontend using HTML/CSS/Bootstrap (or similar), and a database to securely store user accounts, movies, show schedules, and booking records.

The platform focuses on providing an efficient and reliable booking experience by handling essential operations such as real-time seat availability (preventing double booking), booking confirmation, and user booking history. It also includes an **admin-side control panel** (if implemented) to manage movies, theatres/screens, show timings, and monitor bookings. By combining secure session handling, database-backed transactions, and a clean UI, MovieMagic delivers a practical ticket booking solution suitable for real-world usage.

### Scenario 1: Hassle-Free Multi-Mode Travel Booking Experience

MovieMagic allows users to easily find and choose a movie they want to watch. For example, if a user wants to watch a newly released movie, they can open the platform, browse the movie list (or use search/filter options), and select a showtime that fits their schedule. Once a movie is selected, the application fetches show information from the database and displays details like available timings, screen/theatre information, and ticket pricing—making movie selection simple and fast.

### Scenario 2: Seat Selection and Secure Ticket Booking

After selecting a showtime, MovieMagic guides the user to seat selection. The system displays seats based on current availability and prevents already booked seats from being selected again. Once the user chooses seats and confirms the booking, the backend validates availability and records the booking details into the database.

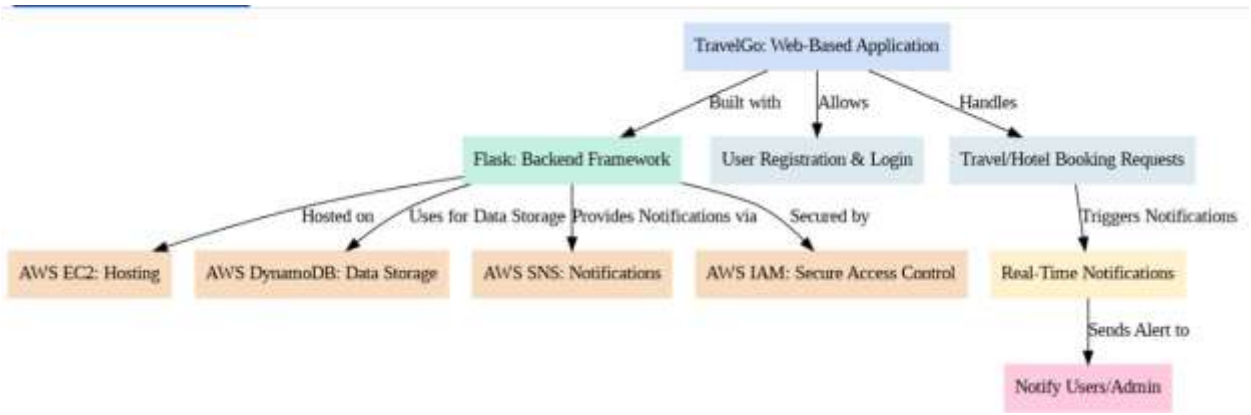
### Scenario 3: User Dashboard with Booking History and Cancellation (If Available)

MovieMagic provides a user dashboard where logged-in users can view their booking history, including:

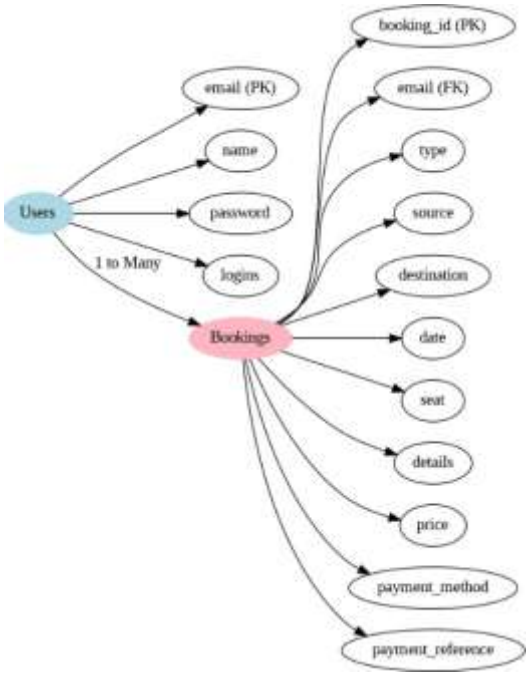
- Movie name
- Show timing
- Number of seats booked
- Booking date/time
- Total price / booking status

If cancellation is implemented, users can cancel eligible bookings directly from the dashboard, and the system updates seat availability and booking status accordingly. This feature improves usability by allowing users to manage their bookings without needing external support.

AWS ARCHITECTURE




Entity Relationship (ER)Diagram:




## Pre-requisites:

1. .**AWS Account Setup**: AWS Account Setup
2. **Understanding IAM**: IAM Overview
3. **Amazon EC2 Basics**: EC2 Tutorial
4. **DynamoDB Basics**: DynamoDB Introduction
5. **SNS Overview**: SNS Documentation
6. **Git Version Control**: Git Documentation

## Project WorkFlow:

### 1. AWS Account Setup and Login

**Activity 1.1:** Set up an AWS account if not already done.

**Activity 1.2:** Log in to the AWS Management Console

### 2. DynamoDB Database Creation and Setup

**Activity 2.1**: Create a DynamoDB Table.

**Activity 2.2**: Configure Attributes for travel-Users and Bookings Tables.

### 3. SNS Notification Setup

Activity 3.1: Create an SNS Topic named BookingConfirmation for sending notifications.

Activity 3.2: Subscribe email to the topic to receive booking and cancellation alerts.

### 4. Backend Development and Application Setup

**Activity 4.1**:Develop the Backend Using Flask.

**Activity 4.2**: Integrate AWS Services Using boto3.

### 5. IAM Role Setup

**Activity 5.1**: Create IAM Role

**Activity 5.2**: Attach Policies

### 6. EC2 Instance Setup

**Activity 6.1**: Launch an EC2 instance to host the Flask application.

**Activity 6.2**: Configure security groups for HTTP, and SSH access.

### 7. Deployment on EC2

**Activity 7.1**:Upload Flask Files

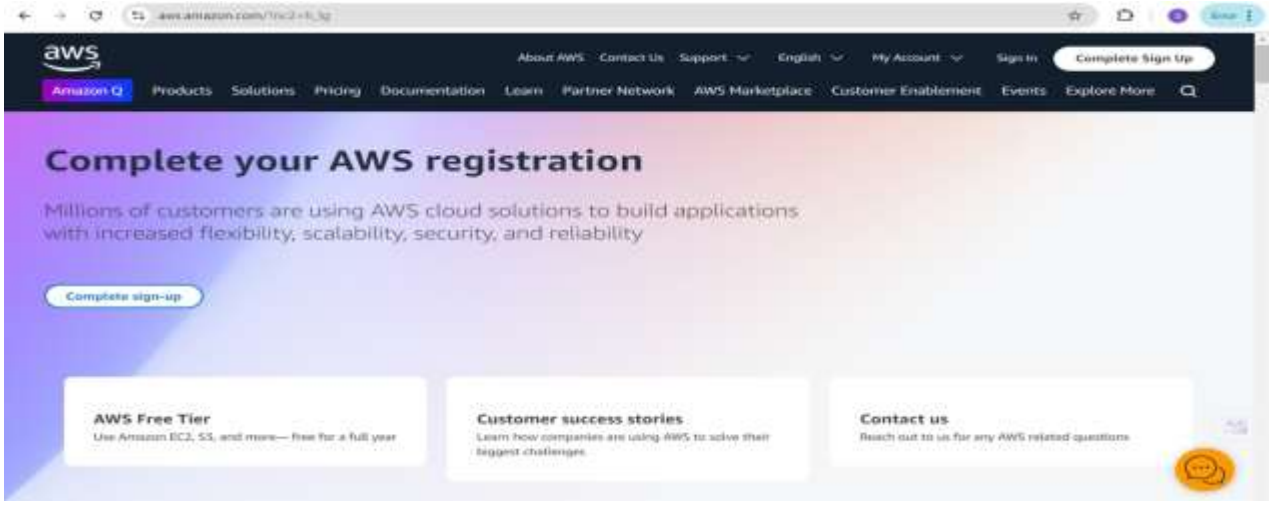
**Activity 7.2**: Run the Flask App

### 8. Testing and Deployment

**Activity 8.1**: Conduct functional testing to verify user registration, login, book requests, and notifications.

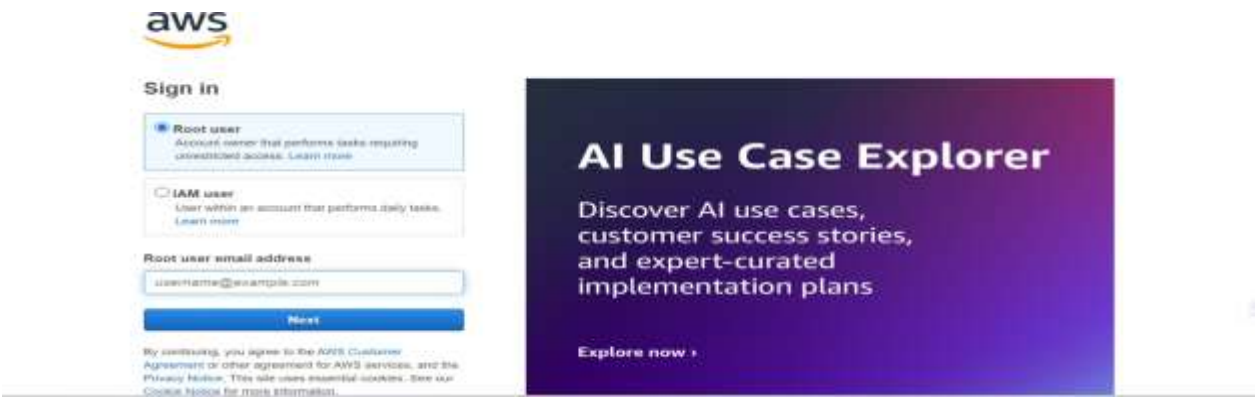
## Milestone 1: AWS Account Setup and Login

- **Activity 1.1: Set up an AWS account if not already done.**
  - Sign up for an AWS account and configure billing settings.



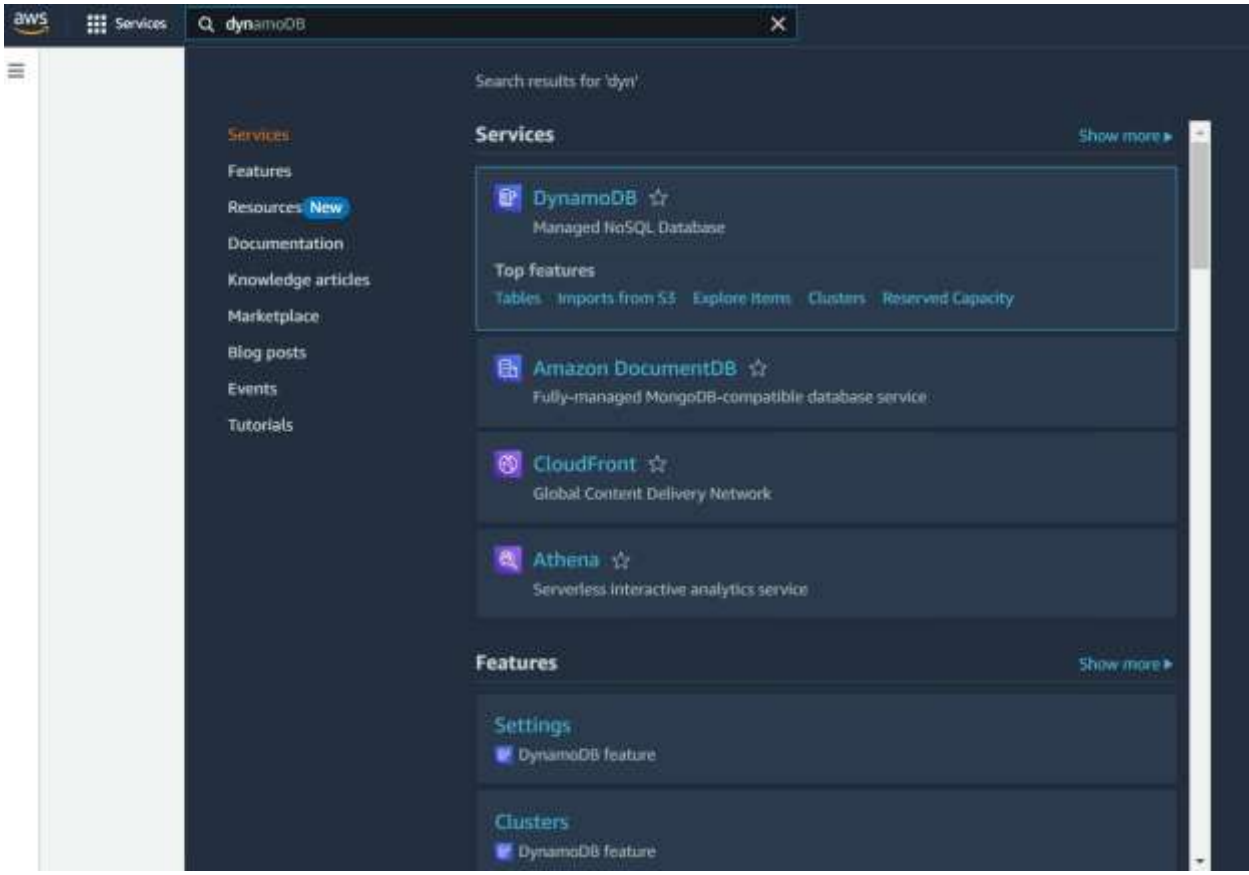

- **Activity 1.2: Log in to the AWS Management Console**

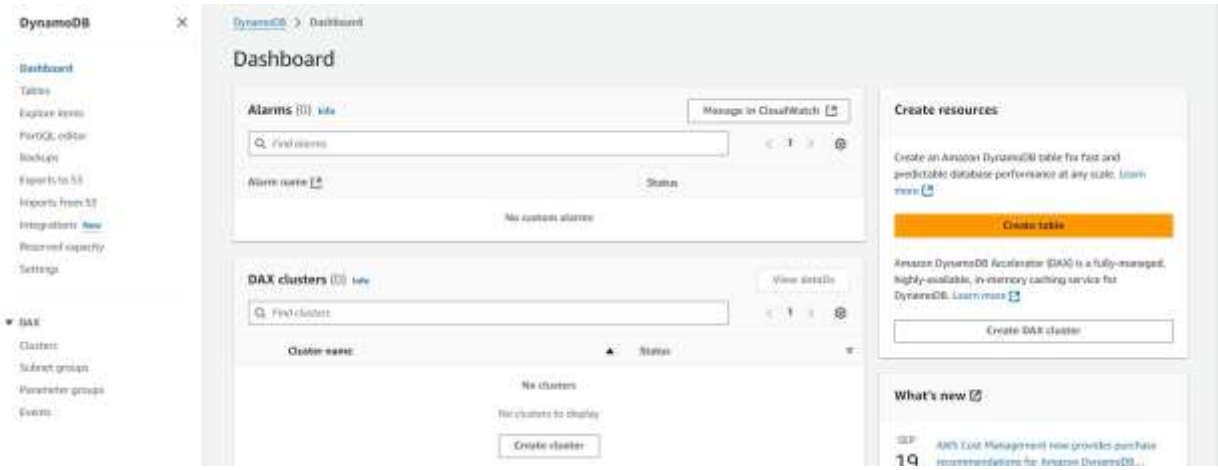
  - After setting up your account, log in to the [AWS Management Console](AWS Management Console).

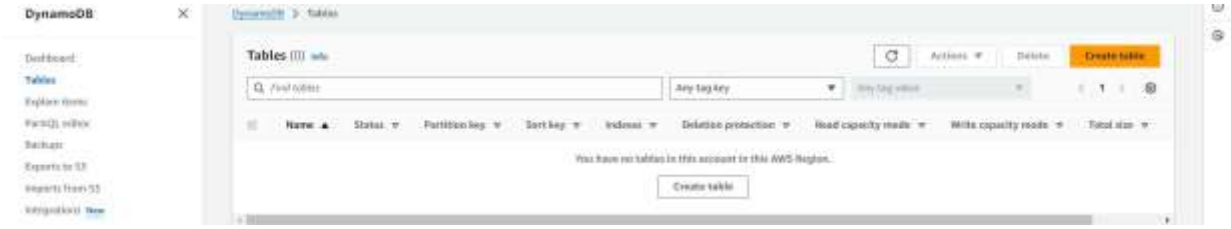



## Milestone 2: DynamoDB Database Creation and Setup

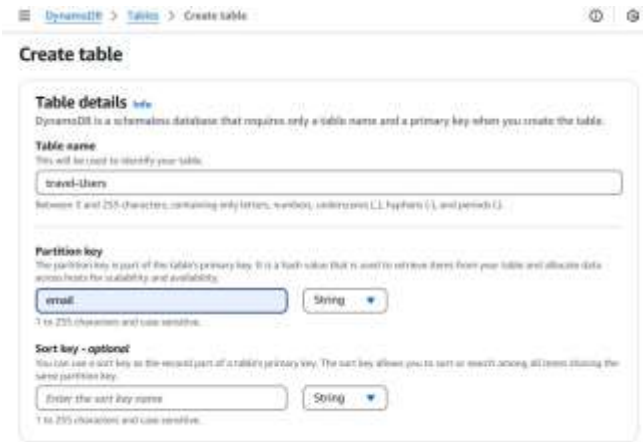
- **Activity 2.1:Navigate to the DynamoDB**

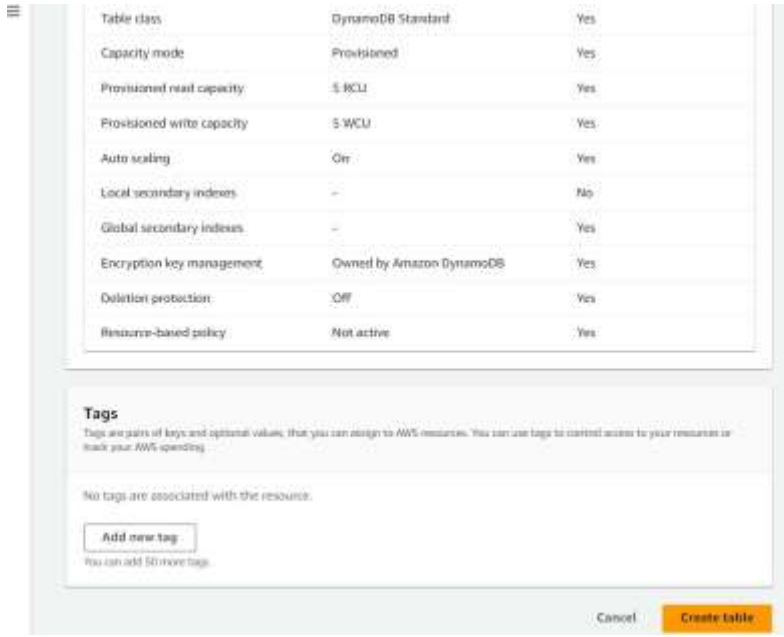
- In the AWS Console, navigate to DynamoDB and click on create tables.
- 

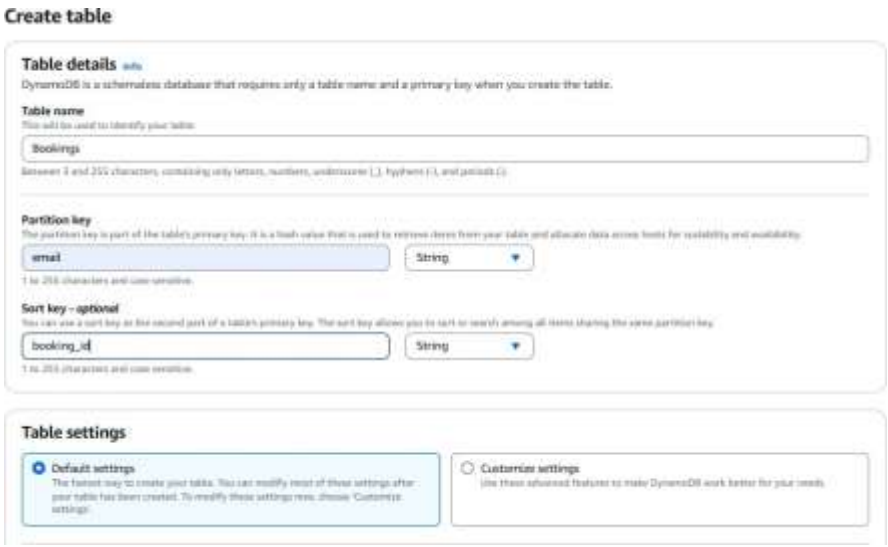



- **Activity 2.2:Create DynamoDB tables for storing registration details and booking records.**
    - Create a travel-Users table for storing registered user details with partition key “Email” with type String and click on create tables.



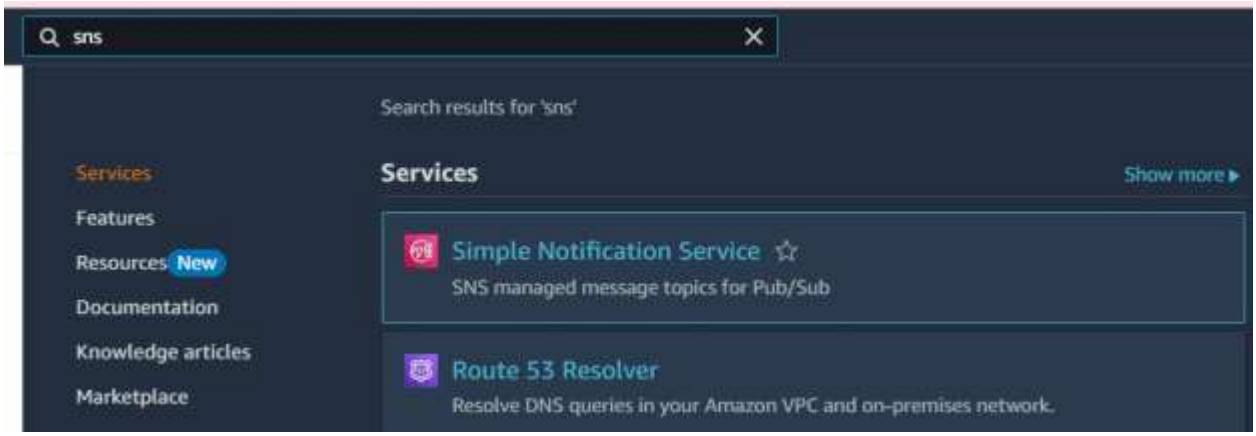



- Follow the same steps to create a `Bookings` for storing booking records with `email` as the partition key and `booking_id` as the sort key.



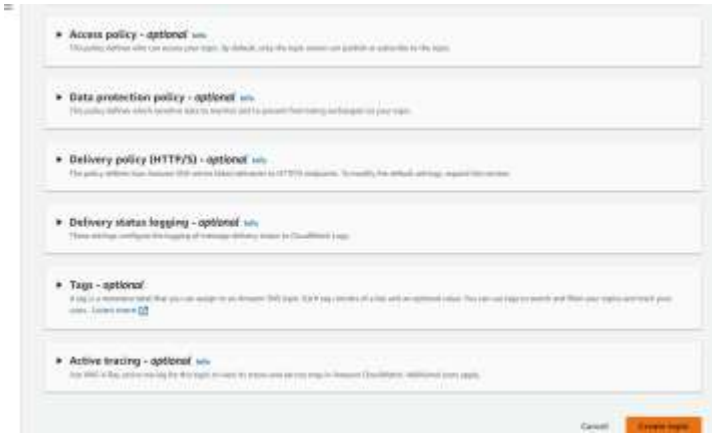
## Milestone 3: SNS Notification Setup

- **Activity 3.1: Create an SNS Topic named BookingConfirmation for sending notifications.**
    - In the AWS Console, search for SNS and navigate to the SNS Dashboard.

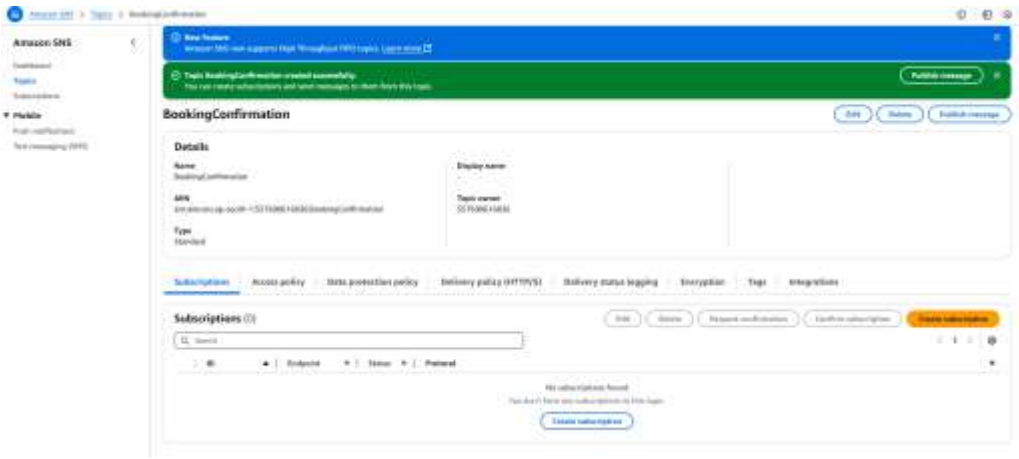


    - Click on **Create Topic** and choose a name for the topic.

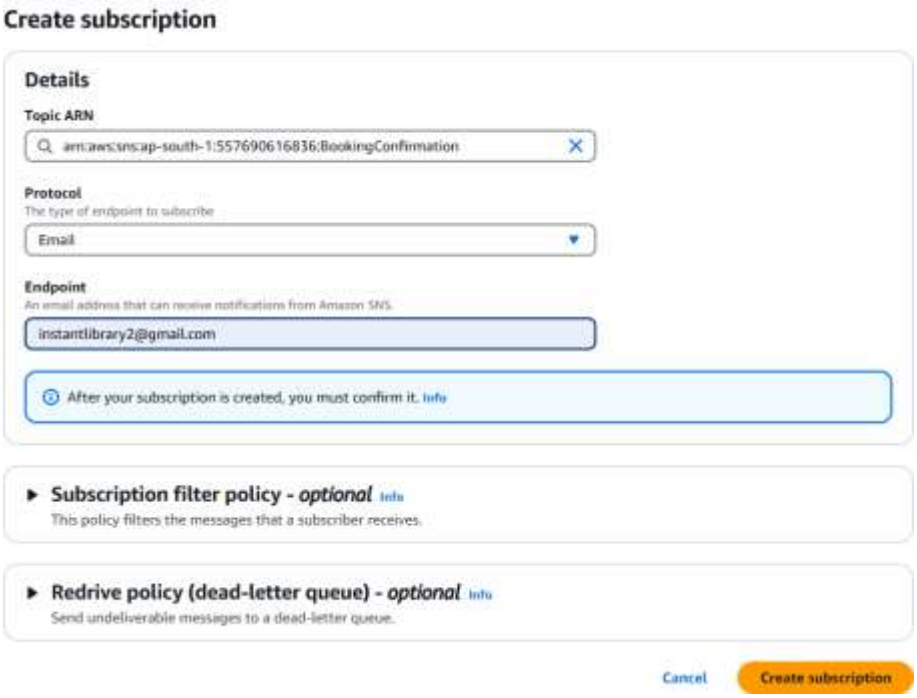
- Choose Standard type for general notification use cases and Click on Create Topic.



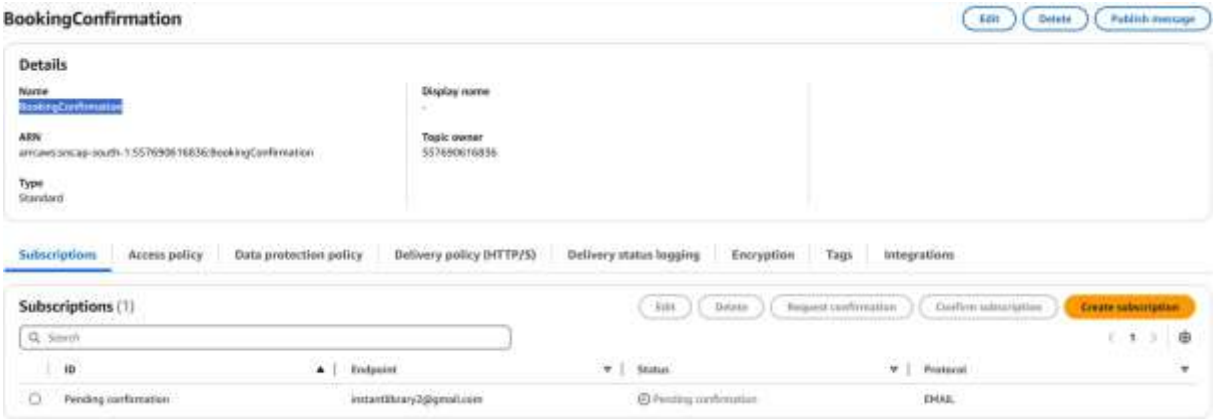

- Configure the SNS topic and note down the **Topic ARN**.



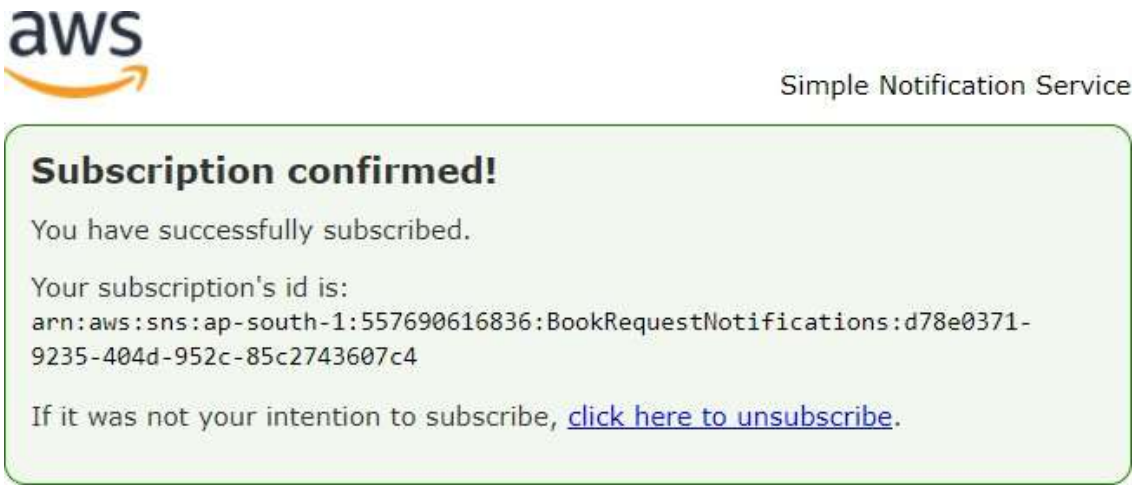
- **Activity 3.2: Subscribe email to the topic to receive booking and cancellation alerts.**
    - Subscribe users (or admin staff) to this topic via Email. When a book request is made, notifications will be sent to the subscribed emails**.**



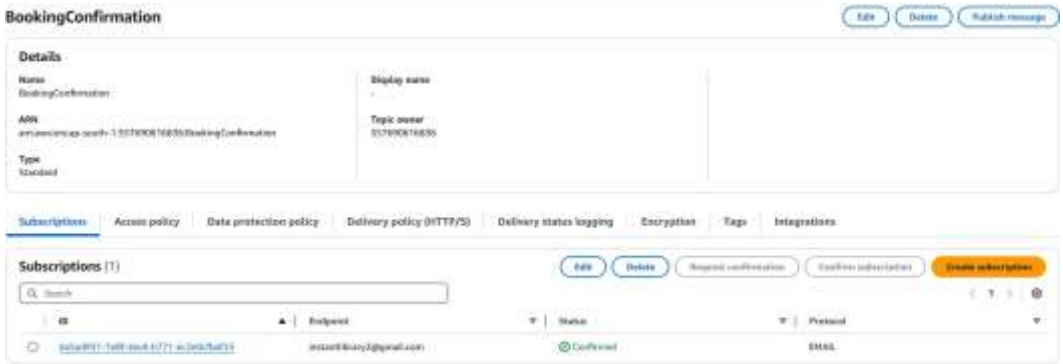

- After subscription request for the mail confirmation



- Navigate to the subscribed Email account and Click on the confirm subscription in the AWS Notification- Subscription Confirmation mail.

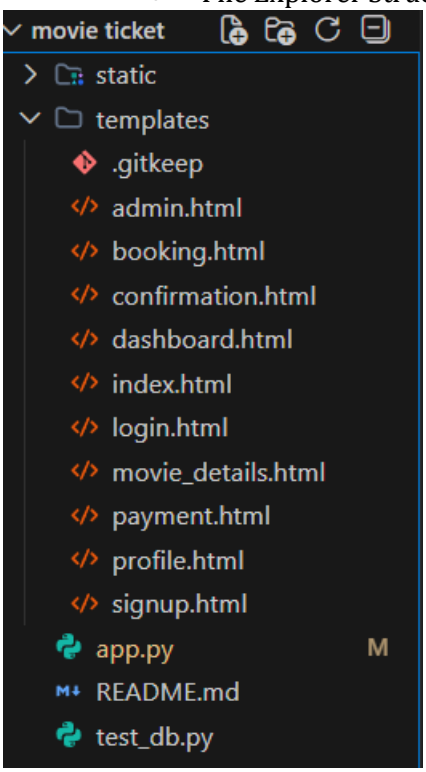



- Successfully done with the SNS mail subscription and setup, now store the ARN link.



# Milestone 4:Backend Development and Application Setup

- **Activity 4.1: Develop the backend using Flask**
  - File Explorer Structure:




Description: Create a `templates` folder containing HTML files for each major feature—bus, train, flight, hotels, seat selection, registration, login, payment, and dashboard—and manage backend logic in `app.py` and `utils/data.py`.

## Description of the code :

- **Flask App Initialization**

  **Import Libraries:**

```
from flask import Flask, render_template, request, redirect, session
import uuid
import boto3
import os
from decimal import Decimal
from boto3.dynamodb.conditions import Attr
from utils.data import transport_data, hotel_data
```

**Description:** Import necessary Flask modules for web routing, Boto3 for AWS DynamoDB access, UUID for unique IDs, and custom transport and hotel data from `utils.data`

**Initialize flask app:**

```
app = Flask(__name__)
```

**Description:** Initialize the Flask application by creating an instance of the `Flask` class with the current module name.

**DynamoDB Connection:**

```
# AWS Setup
dynamodb = boto3.resource('dynamodb', region_name='ap-south-1')


# Tables
users_table = dynamodb.Table('travel-Users')
bookings_table = dynamodb.Table('Bookings')
```

**Description:** Use **boto3** to connect to **DynamoDB** for handling travel-users, booking database operations and also mention region_name where Dynamodb tables are created.

**SNS Connection:**

```
#sns setup

sns = boto3.client('sns', region_name='ap-south-1')
```

**Description:** Configure **SNS** to send notifications when a booking request is submitted.

- **Routes for Web Pages**

**Register User:**

```
@app.route('/')
def home():
    return render_template('index.html', logged_in='user' in session)
```

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        email = request.form['email']
        response = users_table.get_item(Key={'email': email})
        if 'Item' in response:
            return render_template('register.html', message="User already
        users_table.put_item(Item={
            'email': email,
            'name': request.form['name'],
            'password': request.form['password'],
            'logins': 0
        })
        return redirect('/login')
    return render_template('register.html')
```

**Description:** Collecting registration data, and stores user details in the database.

- **login Route (GET/POST)**:

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        response = users_table.get_item(Key={'email': email})
        user = response.get('Item')
        if user and user['password'] == password:
            session['user'] = email
            users_table.update_item(
                Key={'email': email},
                UpdateExpression="ADD logins :inc",
                ExpressionAttributeValues={':inc': Decimal(1)}
            )
            return '''
                <script>
                    localStorage.setItem("loggedIn", "true");
                    window.location.href = "/dashboard";
                </script>
            '''
        return render_template('login.html', message="Invalid credentials.
    return render_template('login.html')
```

**Description:** Verifies user credentials, increments login count, and redirects to the dashboard on success.

**Dashboard route:**

```
@app.route('/dashboard')
def dashboard():
    if 'user' not in session:
        return redirect('/login')
    email = session['user']
    user = users_table.get_item(Key={'email': email}).get('Item')
    response = bookings_table.scan(FilterExpression=Attr('user_email').eq(email))
    return render_template('dashboard.html', name=user['name'], bookings=response['Items'])
```

**Description:** Define the `/dashboard` route to display the user's booking data from DynamoDB after verifying the session and fetching user details using their email.

**Seat Routes:**

```
# ---------- Transport Pages ----------
@app.route('/bus', methods=['GET', 'POST'])
def bus():
    if request.method == 'POST':
        s, d, dt = request.form['source'], request.form['destination'], request.form['date']
        buses = [b for b in transport_data['bus'] if b['source'] == s and b['destination'] == d]
        return render_template('bus.html', buses=buses, source=s, destination=d, date=dt)
    return render_template('bus.html', buses=None)

@app.route('/train', methods=['GET', 'POST'])
def train():
    if request.method == 'POST':
        s, d, dt = request.form['source'], request.form['destination'], request.form['date']
        trains = [t for t in transport_data['train'] if t['source'] == s and t['destination'] == d]
        return render_template('train.html', trains=trains, source=s, destination=d, date=dt)
    return render_template('train.html', trains=None)

@app.route('/flight', methods=['GET', 'POST'])
def flight():
    if request.method == 'POST':
        s, d, dt = request.form['source'], request.form['destination'], request.form['date']
        flights = [f for f in transport_data['flight'] if f['source'] == s and f['destination'] == d]
        return render_template('flight.html', flights=flights, source=s, destination=d, date=dt)
    return render_template('flight.html', flights=None)

@app.route('/hotels', methods=['GET', 'POST'])
def hotels():
    if request.method == 'POST':
        city = request.form['city']
        hotels = [h for h in hotel_data if h['location'] == city]
        return render_template('hotels.html', hotels=hotels, city=city)
    return render_template('hotels.html', hotels=None)
```

```python
@app.route('/select_seats')
def select_seats():
    return render_template("select_seats.html")
```

**Description:** Define separate route handlers for bus, train, flight, and hotel search functionalities using POST requests to filter results from local datasets based on user input like source, destination, date, or city and also add seat selecting routes for bus page..

**Ticket routes:**

```python
# ---------- Booking ----------
@app.route('/book', methods=['POST'])
def book():
    if 'user' not in session:
        return redirect('/login')

    session['pending_booking'] = {
        "booking_id": str(uuid.uuid4())[:8],
        "type": request.form['type'],
        "source": request.form['source'],
        "destination": request.form['destination'],
        "date": request.form['date'],
        "seat": request.form.get('seat', 'N/A'),
        "details": request.form['details'],
        "price": Decimal(request.form['price']),
        "user_email": session['user'],
        "email": session['user']
    }
    return render_template("payment.html", booking=session['pending_booking'])
```

**Description:** Saves booking details in session and redirects to the payment page if the user is logged in.

**Payment Route:**

```
@app.route('/payment', methods=['POST'])
def payment():
    if 'user' not in session or 'pending_booking' not in session:
        return redirect('/login')

    booking = session.pop('pending_booking')
    booking['payment_method'] = request.form['method']
    booking['payment_reference'] = request.form['reference']

    bookings_table.put_item(Item=booking)

    # SNS Booking Notification
    try:
        sns.publish(
            TopicArn=os.getenv('SNS_TOPIC_ARN'),
            Message=f"✅ Booking Confirmed!\n{booking['details']} on {booking['date']} for ₹{booking['price']}",
            Subject="TravelGo Booking Confirmation"
        )
    except Exception as e:
        print("SNS Error (booking):", e)

    return redirect('/dashboard')
```

**Description:** Processes payment details, stores the booking in the database, and sends an SNS confirmation notification before redirecting to the dashboard.

**Remove Booking route:**

```
@app.route('/remove_booking', methods=['POST'])
def remove_booking():
    if 'user' not in session:
        return redirect('/login')
    email = session['user']
    booking_id = request.form.get('booking_id')

    try:
        bookings_table.delete_item(Key={'email': email, 'booking_id': booking_id})
        # SNS Cancellation Notification
        sns.publish(
            TopicArn=os.getenv('SNS_TOPIC_ARN'),
            Message=f"❌ Booking Canceled\nBooking ID: {booking_id}",
            Subject="TravelGo Cancellation"
        )
    except Exception as e:
        print("Delete Error:", e)

    return redirect('/dashboard')
```

**Description:**Deletes a booking from the database and sends a cancellation notification via SNS, then redirects to the dashboard.

**Logout route:**

```
@app.route('/logout')
def logout():
    session.clear()
    return '''
        <script>
            localStorage.setItem("loggedIn", "false");
            window.location.href = "/";
        </script>
    '''
```

**Description:** Implement the logout route to clear session data and reset `loggedIn` in localStorage before redirecting the user to the homepage.

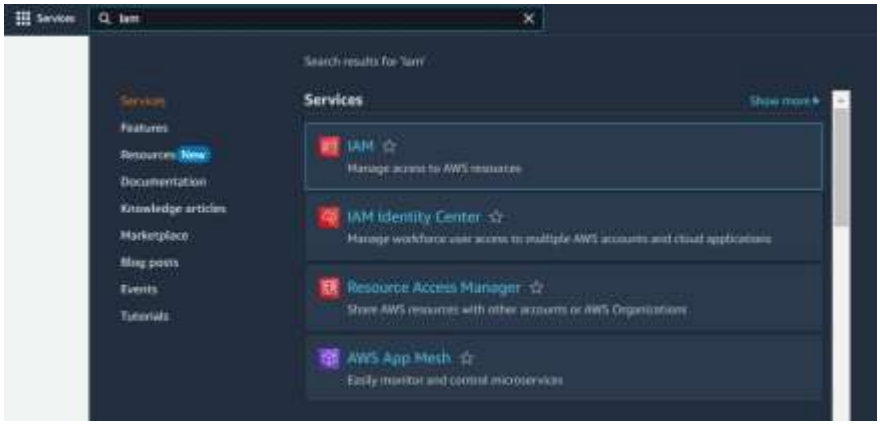
**Deployment Code:**

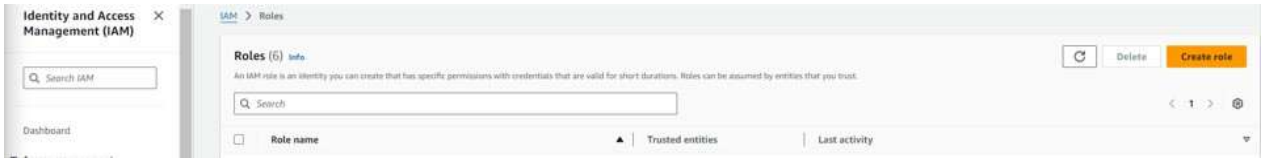
```
if __name__ == "__main__":
    app.run(host='0.0.0.0', port=80, debug=True)
```

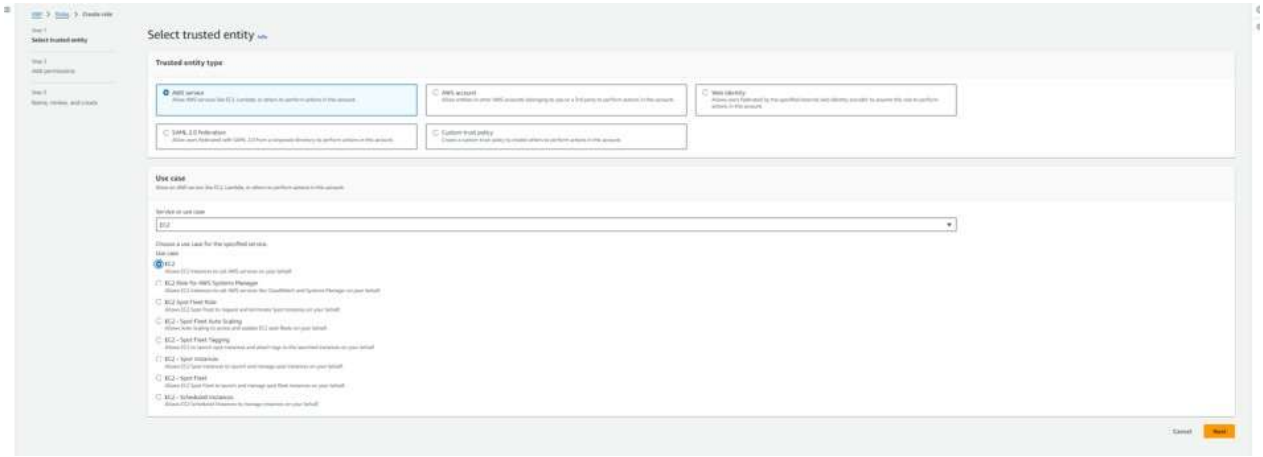
**Description:** Start the Flask application on all network interfaces at port 80 with debug mode enabled for easier development and testing.

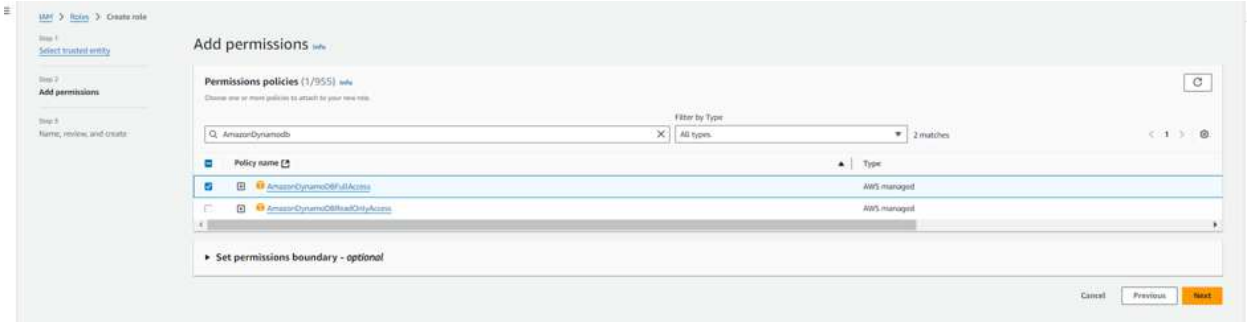
## Milestone 5: IAM Role Setup

- **Activity 5.1:Create IAM Role.**
    - In the AWS Console, go to IAM and create a new IAM Role for EC2 to interact with DynamoDB and SNS.





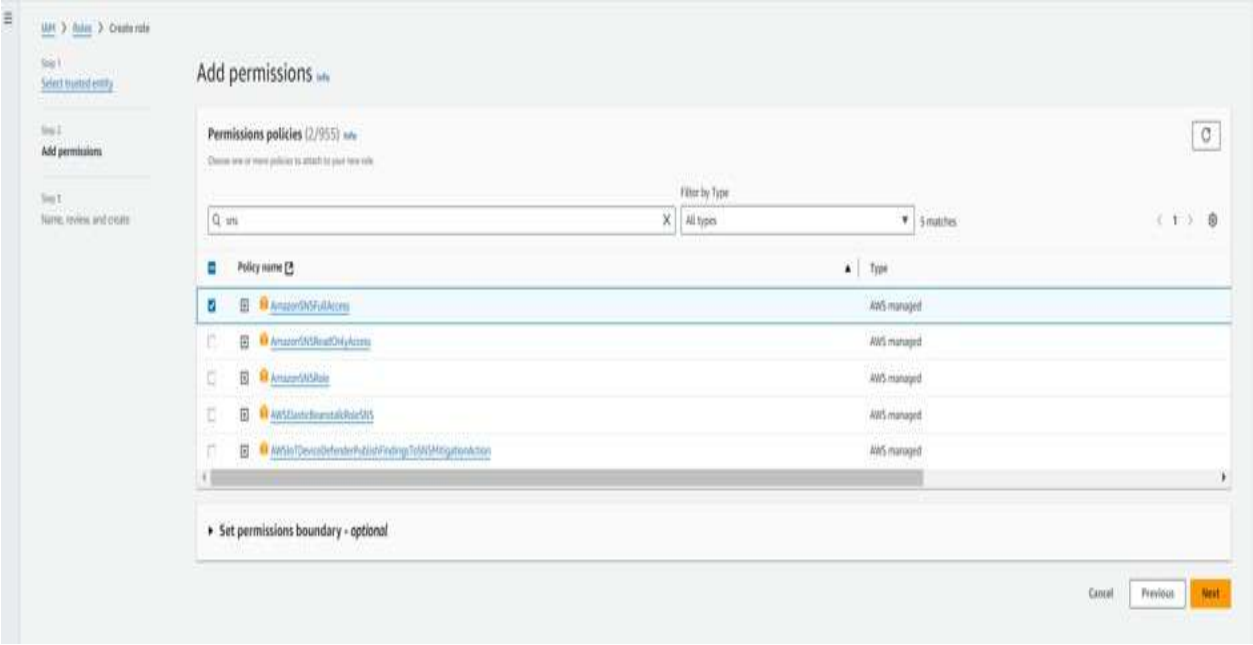



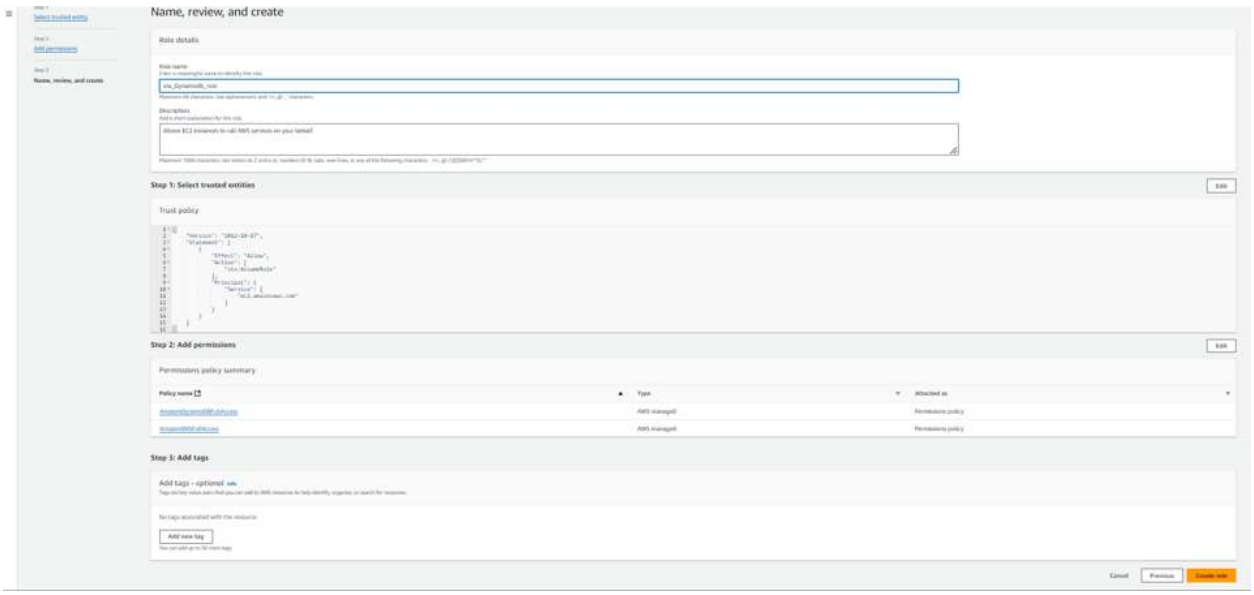



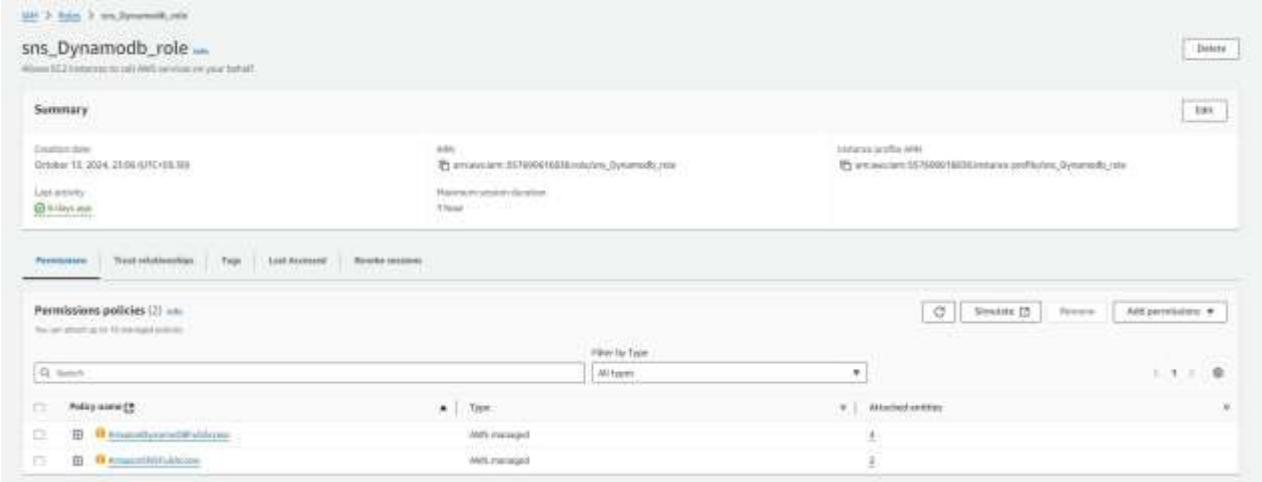

- **Activity 5.2: Attach Policies.**

  Attach the following policies to the role:

  - AmazonDynamoDBFullAccess: Allows EC2 to perform read/write operations on DynamoDB.
  - AmazonSNSFullAccess: Grants EC2 the ability to send notifications via SNS.

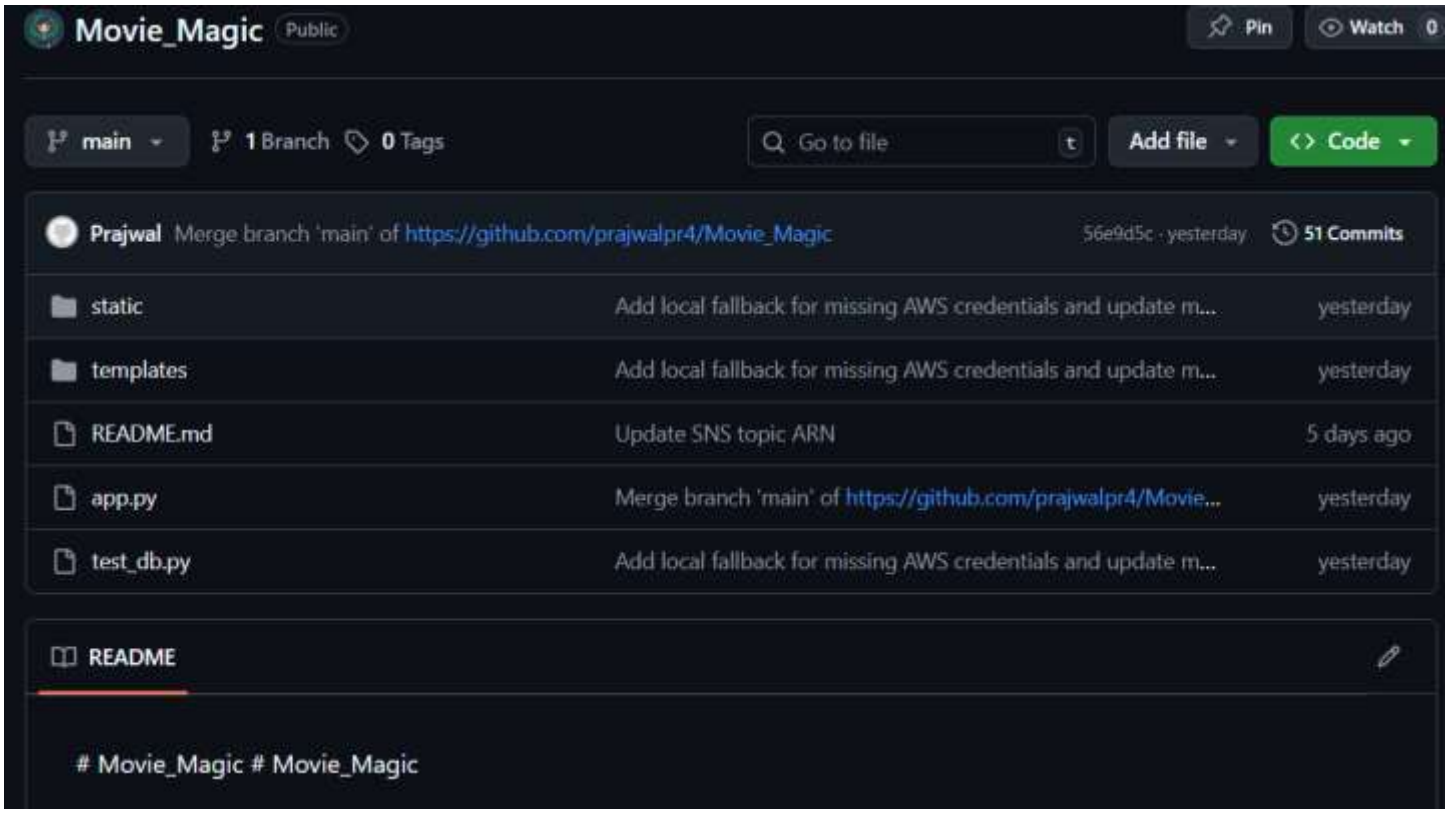

IAM > Roles > Create role
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Add permissions Info
Permissions policies (2/955) Info
Choose one or more policies to attach to your new role.
Filter by Type
sns
All types
5 matches
Policy name
Type
AmazonSNSFullAccess
AWS managed
AmazonSNSReadOnlyAccess
AWS managed
AmazonSNSRole
AWS managed
AWSElasticBeanstalkRoleSNS
AWS managed
AWSIoTDeviceDefenderPublishFindingsToSNSMitigationAction
AWS managed
Set permissions boundary - optional
Cancel
Previous
Next

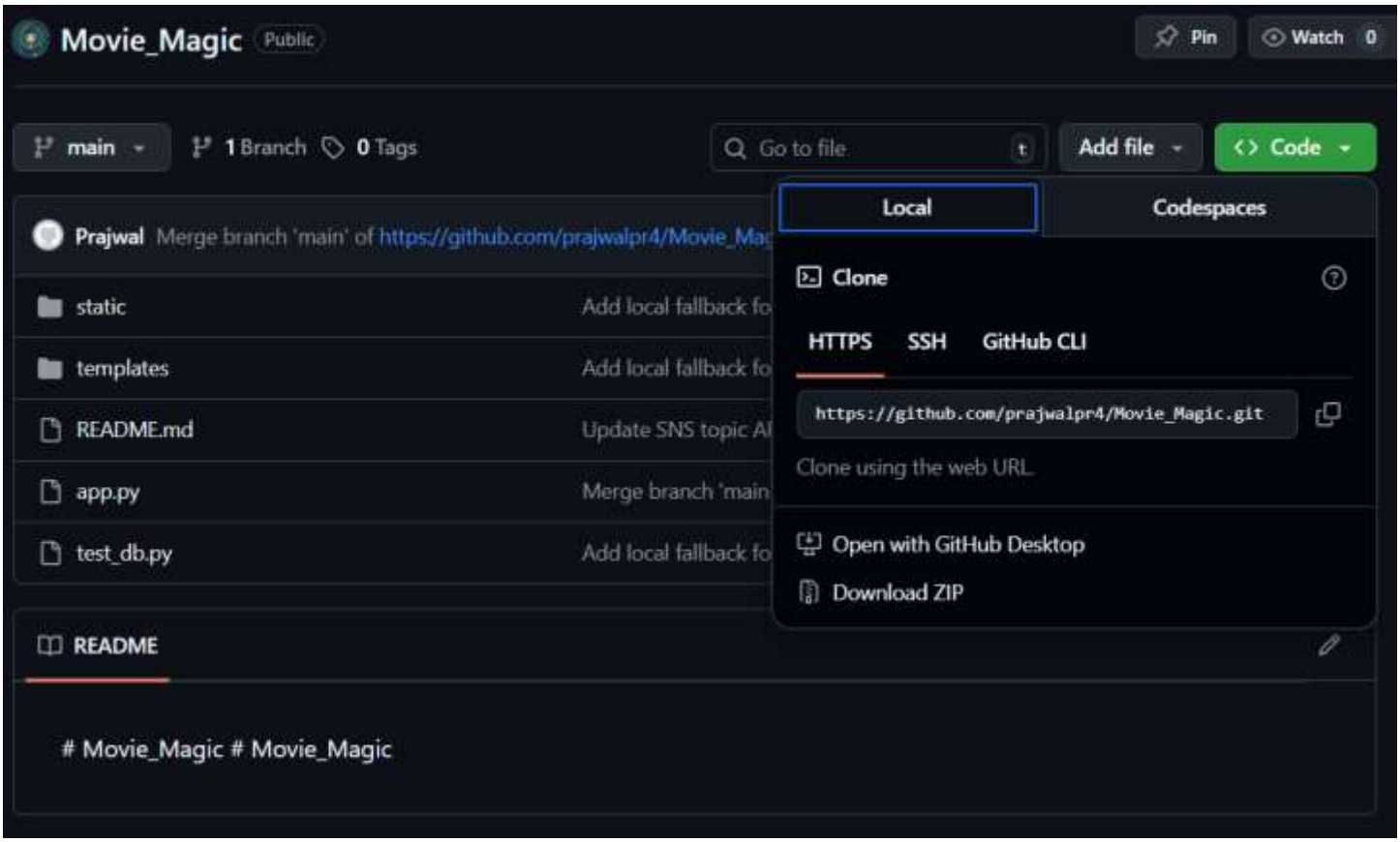

Name, review, and create
Role details
Step 1: Select trusted entities
Trust policy
Step 2: Add permissions
Permissions policy summary
Policy name
Type
Attached as
Step 3: Add tags
Add tags - optional
Add new tag
Cancel
Previous
Create role




## Milestone 6: EC2 Instance Setup

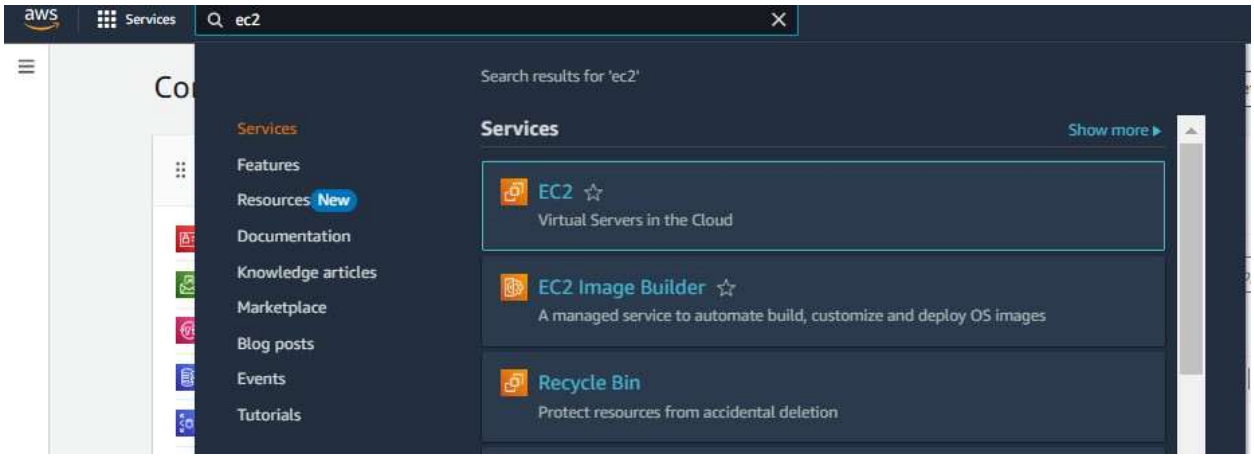
- **Note: Load your Flask app and Html files into GitHub repository.**





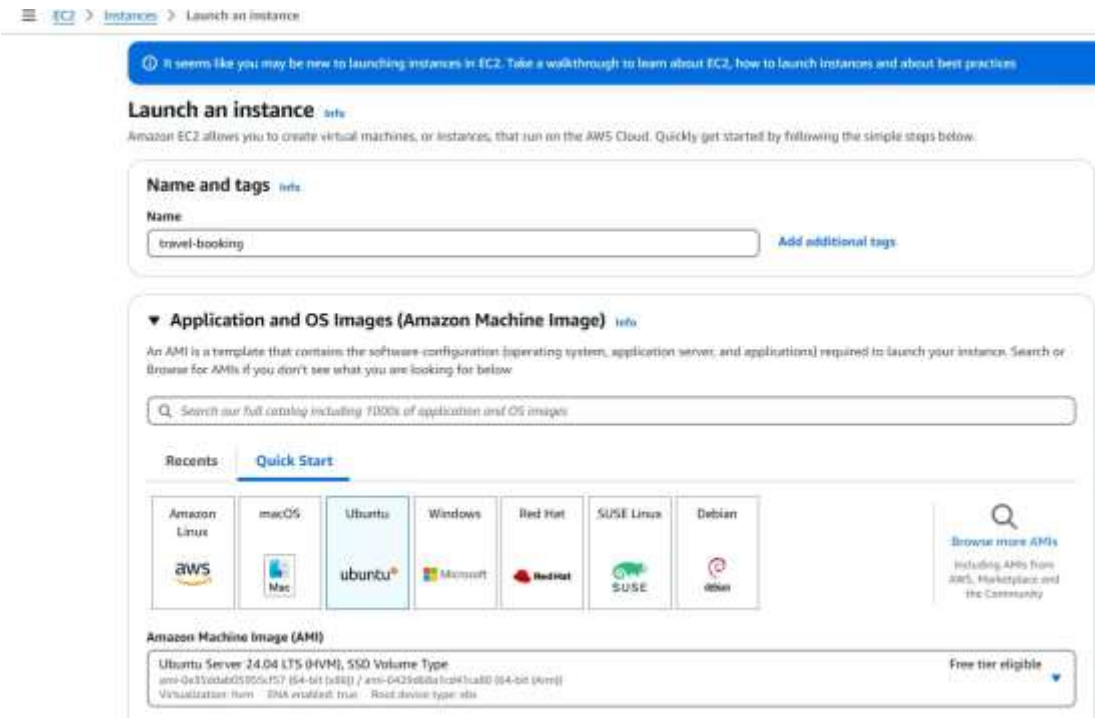

- **Activity 6.1: Launch an EC2 instance to host the Flask application.**

  - **Launch EC2 Instance**
    - In the AWS Console, navigate to EC2 and launch a new instance.

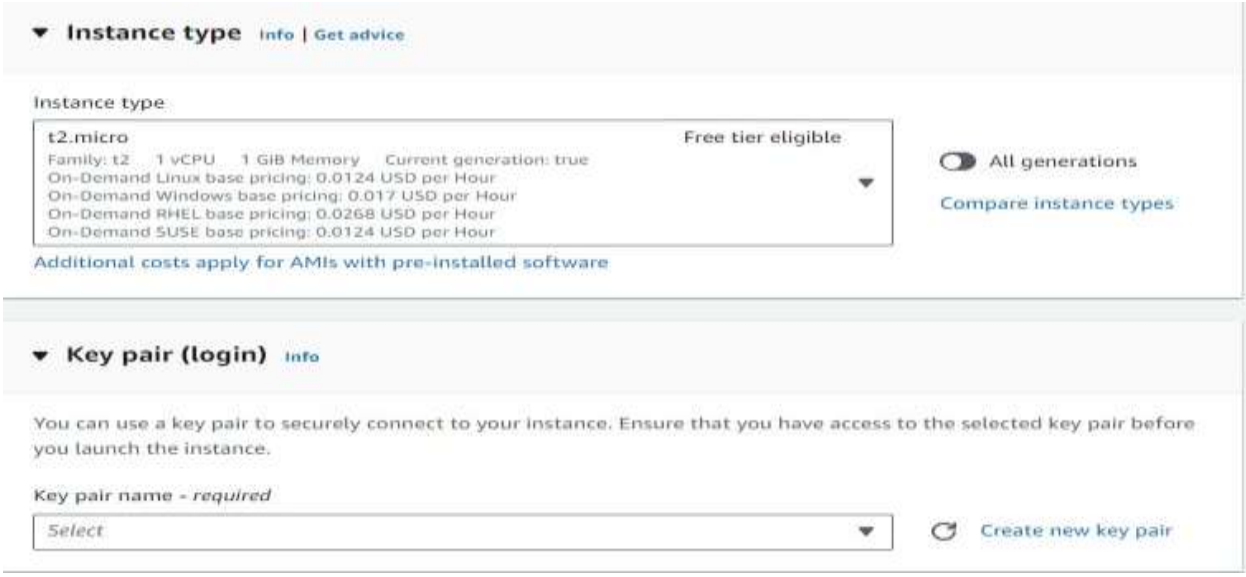


- Click on Launch instance to launch EC2 instance

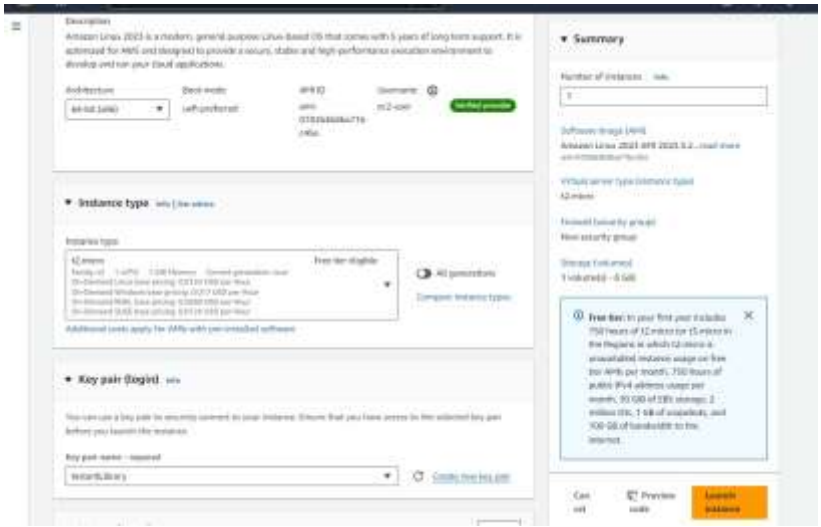
- Choose Amazon Linux 2 or Ubuntu as the AMI and t2.micro as the instance type (free-tier eligible).



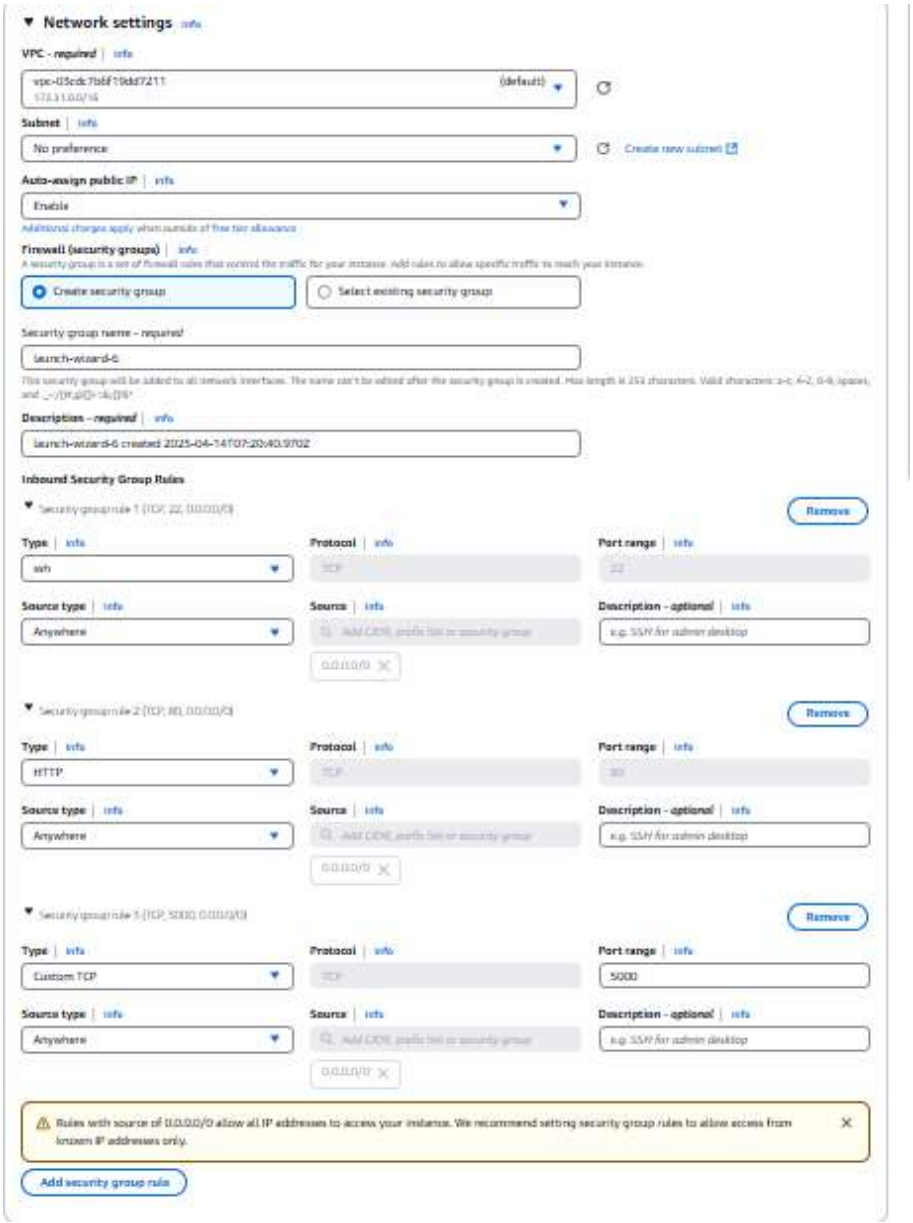

- Create and download the key pair for Server access.

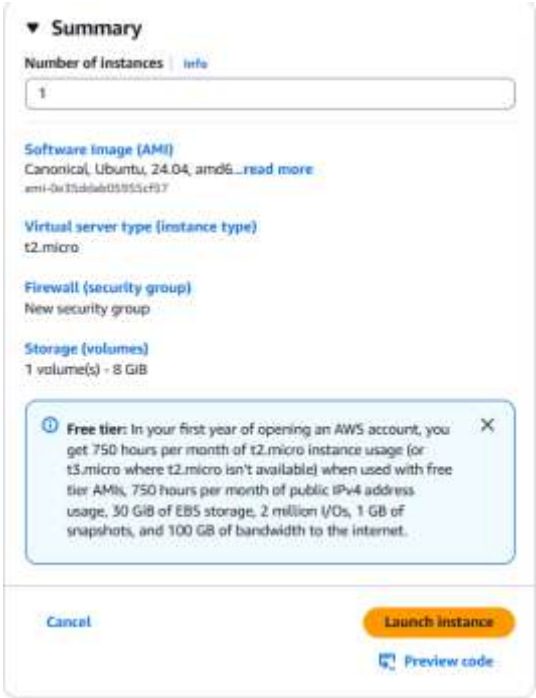


InstantLibrary.pem



- **Activity 6.2:Configure security groups for HTTP, and SSH access.**

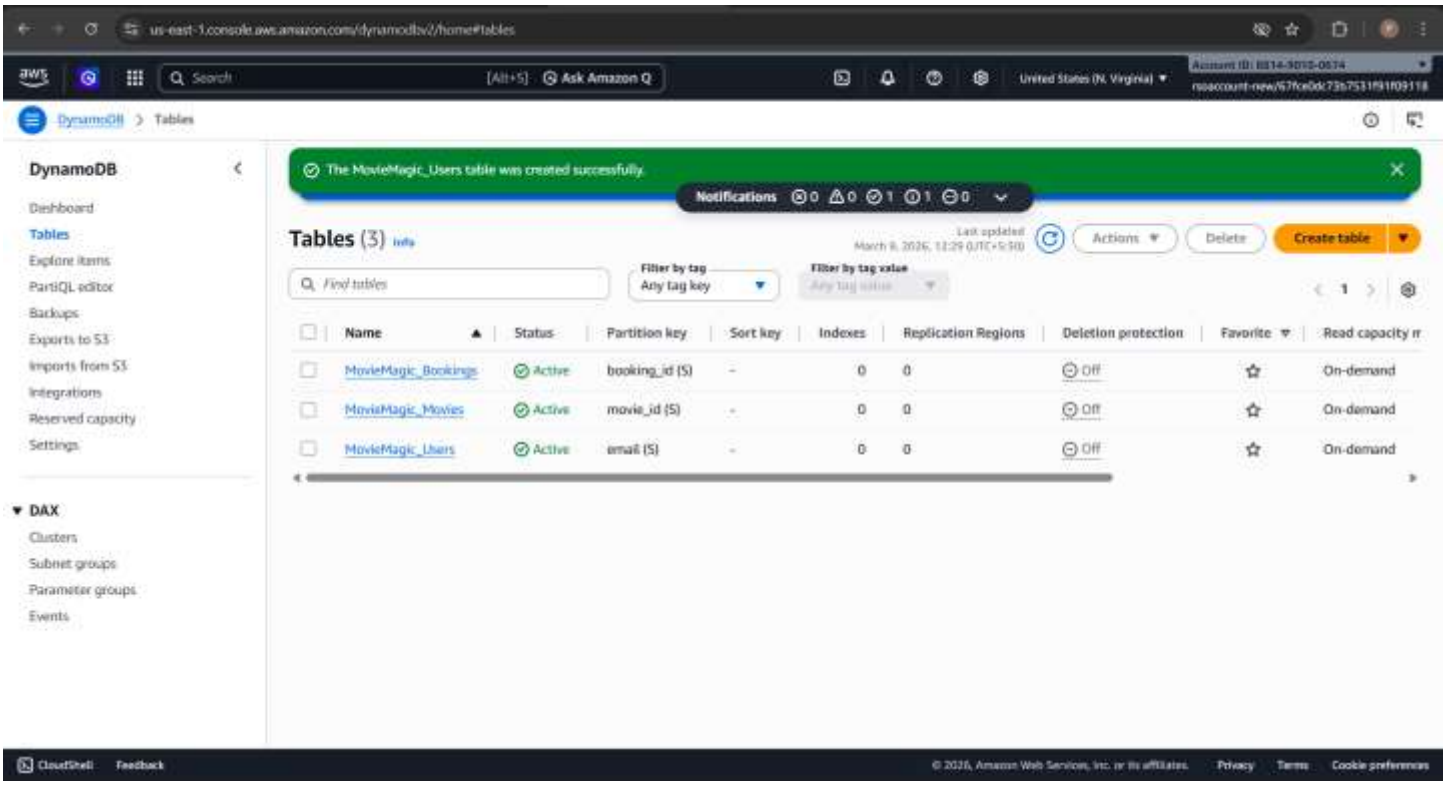




- To connect to EC2 using **EC2 Instance Connect**, start by ensuring that an **IAM role** is attached to your EC2 instance. You can do this by selecting your instance, clicking on **Actions**, then navigating to **Security** and selecting **Modify IAM Role** to attach the appropriate role. After the IAM role is connected, navigate to the **EC2** section in the **AWS Management Console**. Select the **EC2 instance** you wish to connect to. At the top of the **EC2 Dashboard**, click the **Connect** button. From the connection methods presented, choose **EC2 Instance Connect**. Finally, click **Connect** again, and a new browser-based terminal will open, allowing you to access your EC2 instance directly from your browser.

SMARTBRIDGE
Let's Bridge the Gap

Smart Internz



- Now connect the EC2 with the files

```
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1024-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Tue Apr 15 15:33:31 UTC 2025

  System load:  0.0               Processes:             105
  Usage of /:   36.2% of 6.71GB   Users logged in:       0
  Memory usage: 20%               IPv4 address for enX0: 172.31.12.110
  Swap usage:   0%

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

63 updates can be applied immediately.
33 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


Last login: Tue Apr 15 15:33:32 2025 from 13.233.177.5
ubuntu@ip-172-31-12-110:~$
```

# Milestone 7: Deployment on EC2

## Activity 7.1: Install Software on the EC2 Instance

**# Update and install Python + pip**

sudo apt update

sudo apt install python3-pip -y

sudo apt install git -y

sudo apt install python3-venv -y

python3 -m venv venv

source venv/bin/activate

## Activity 7.2:Clone Your Flask Project from GitHub

**Clone your project repository from GitHub into the EC2 instance using Git.**

Run: 'git clone https://github.com/prajwalpr4/Movie_Magic
Note: change your-github-username and your-repository-name with your credentials

here: 'https://github.com/prajwalpr4/Movie_Magic'

- This will download your project to the EC2 instance.

## To navigate to the project directory, run the following command:

cd <your-folder>

## Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

### Run the Flask Application

export SNS_TOPIC_ARN=arn:aws:sns:ap-south-1:557690616836:BookingConfirmation

git pull origin main ( Only if you made changes in git and want to reflect them in the EC2 terminal.)

**# Install requirements**

pip install flask boto3

**Launch the Flask app:**

sudo -E venv/bin/python3 app.py

**Verify the Flask app is running**:

http://your-ec2-public-ip

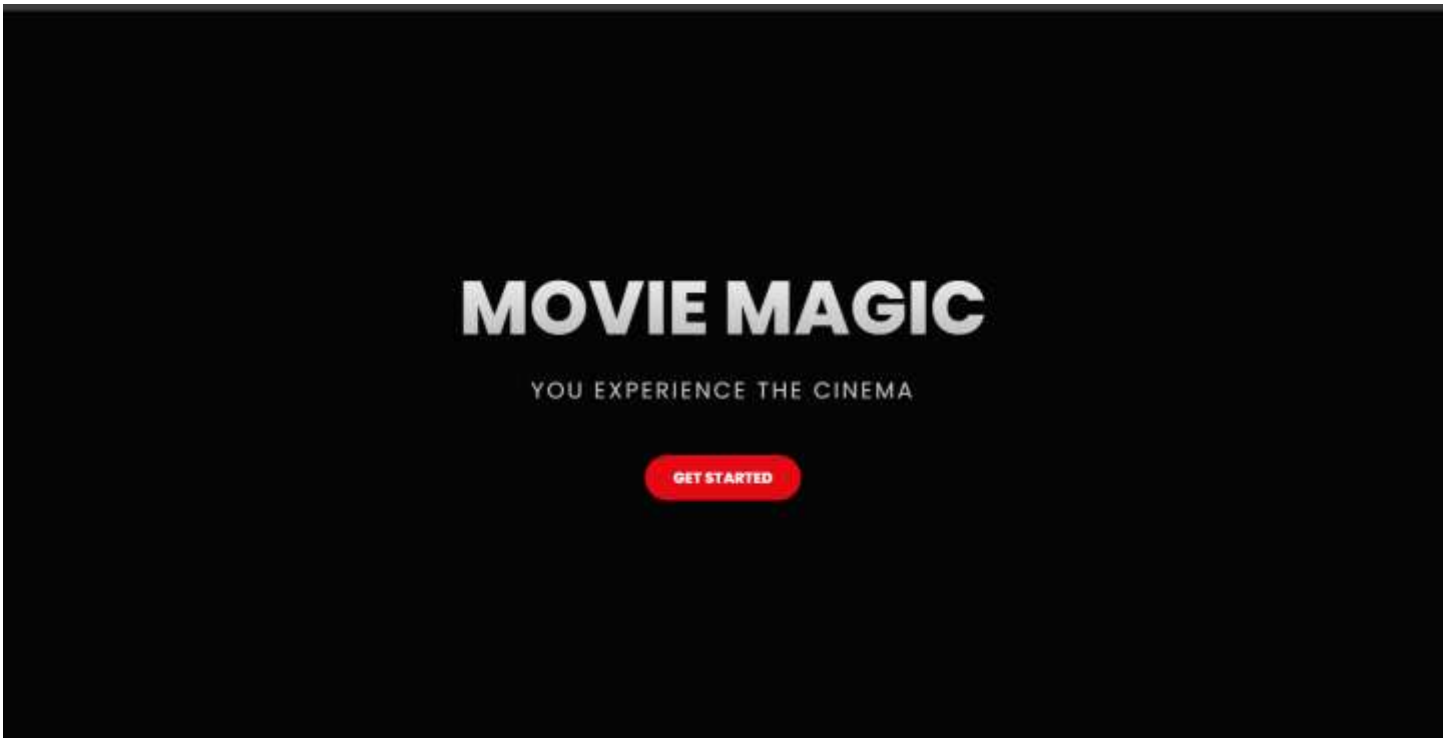
- Run the Flask app on the EC2 instance

```
Last login: Mon Apr 14 16:27:44 2025 from 13.233.177.5
ubuntu@ip-172-31-12-110:~$ cd Travel-Booking-website_Ec2_Dynamodb_SNS
ubuntu@ip-172-31-12-110:~/Travel-Booking-website_Ec2_Dynamodb_SNS$ cd travel_booking_system
ubuntu@ip-172-31-12-110:~/Travel-Booking-website_Ec2_Dynamodb_SNS/travel_booking_system$ export SNS_TOPIC_ARN=arn:aws:sns:ap-south-1:557690616036:BookingConfirmation
ubuntu@ip-172-31-12-110:~/Travel-Booking-website_Ec2_Dynamodb_SNS/travel_booking_system$ git pull origin main
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 1.68 KiB | 429.00 KiB/s, done.
From https://github.com/AlekhyaPenubakula/Travel-Booking-website_Ec2_Dynamodb_SNS
 * branch            main       -> FETCH_HEAD
   2289f47..397e3a1  main       -> origin/main
Updating 2289f47..397e3a1
Fast-forward
 travel_booking_system/templates/hotels.html | 107 ++++++
```

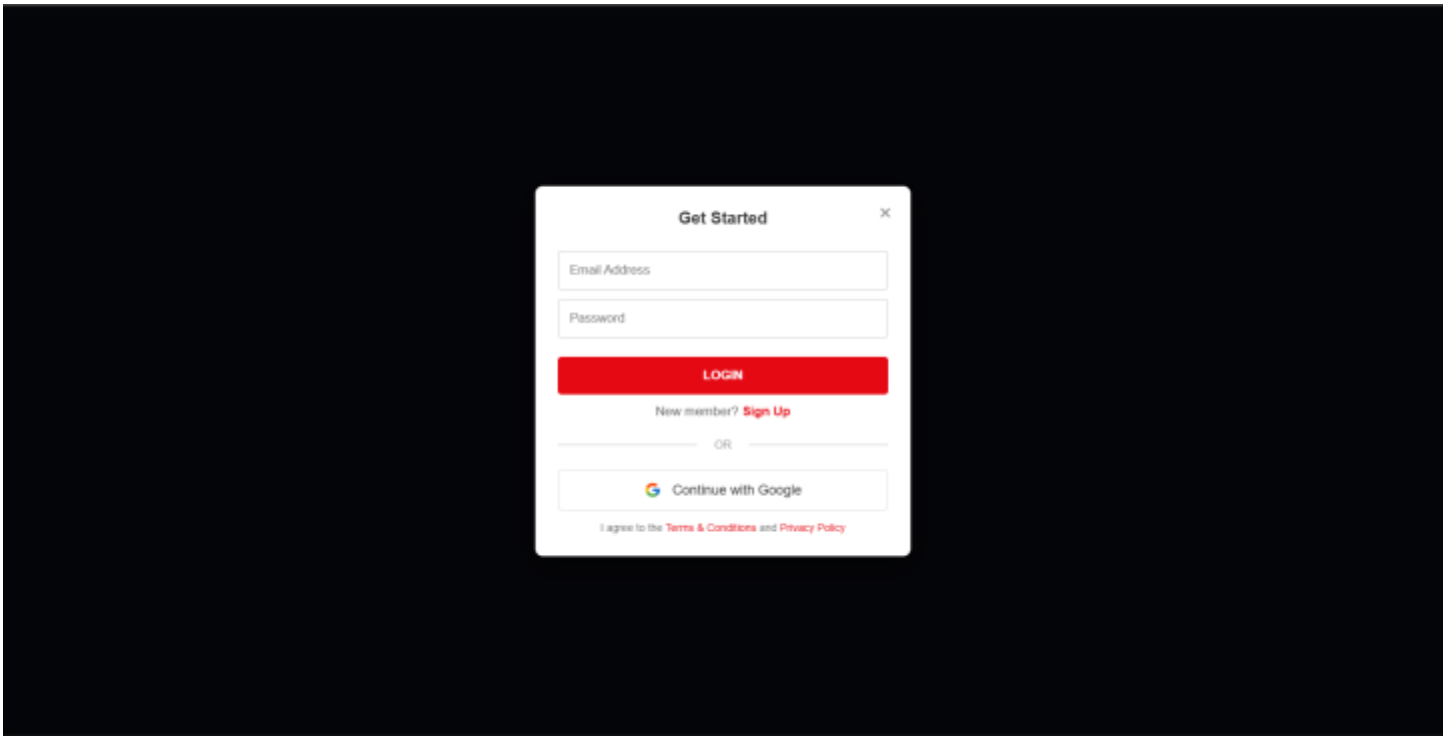
## Milestone 8: Testing and Deployment

- **Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.**

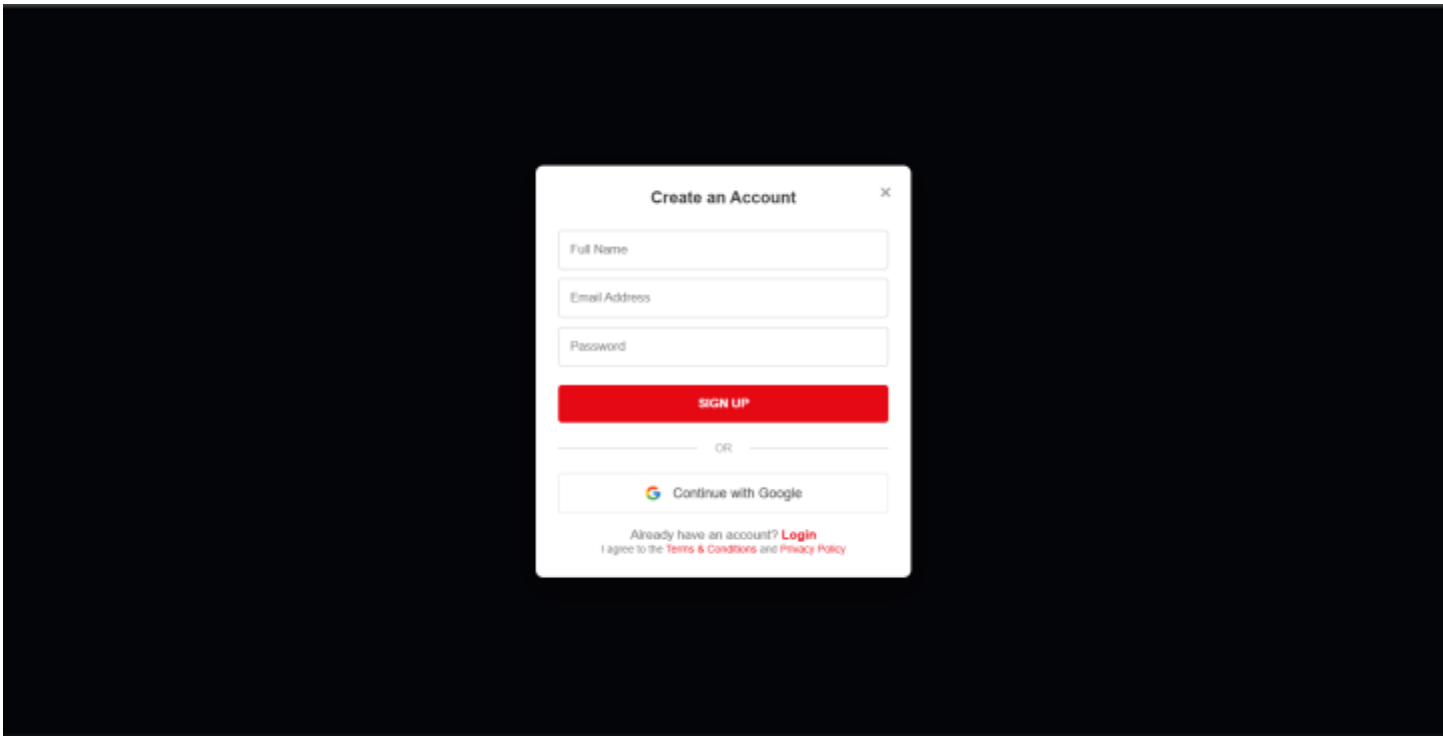
**Home Page:**



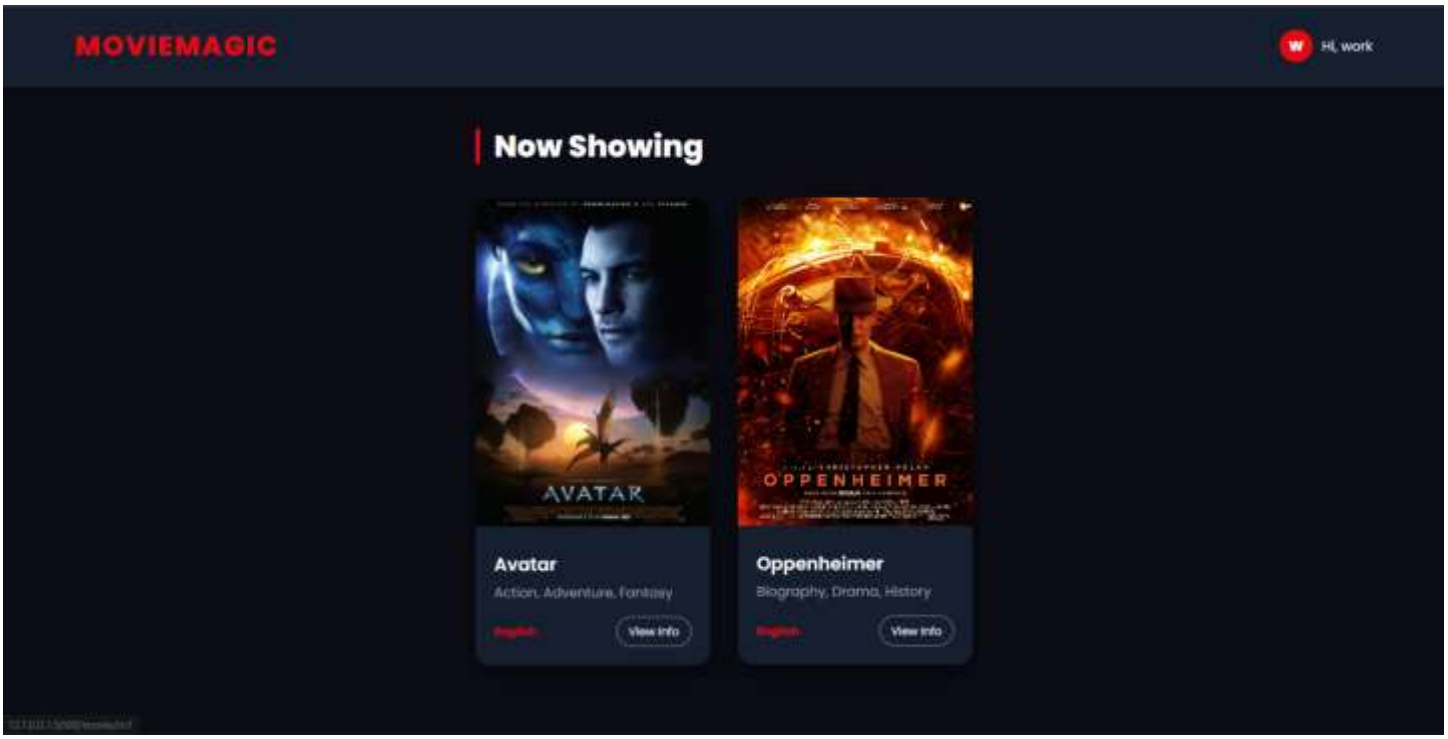
**Login Page:**



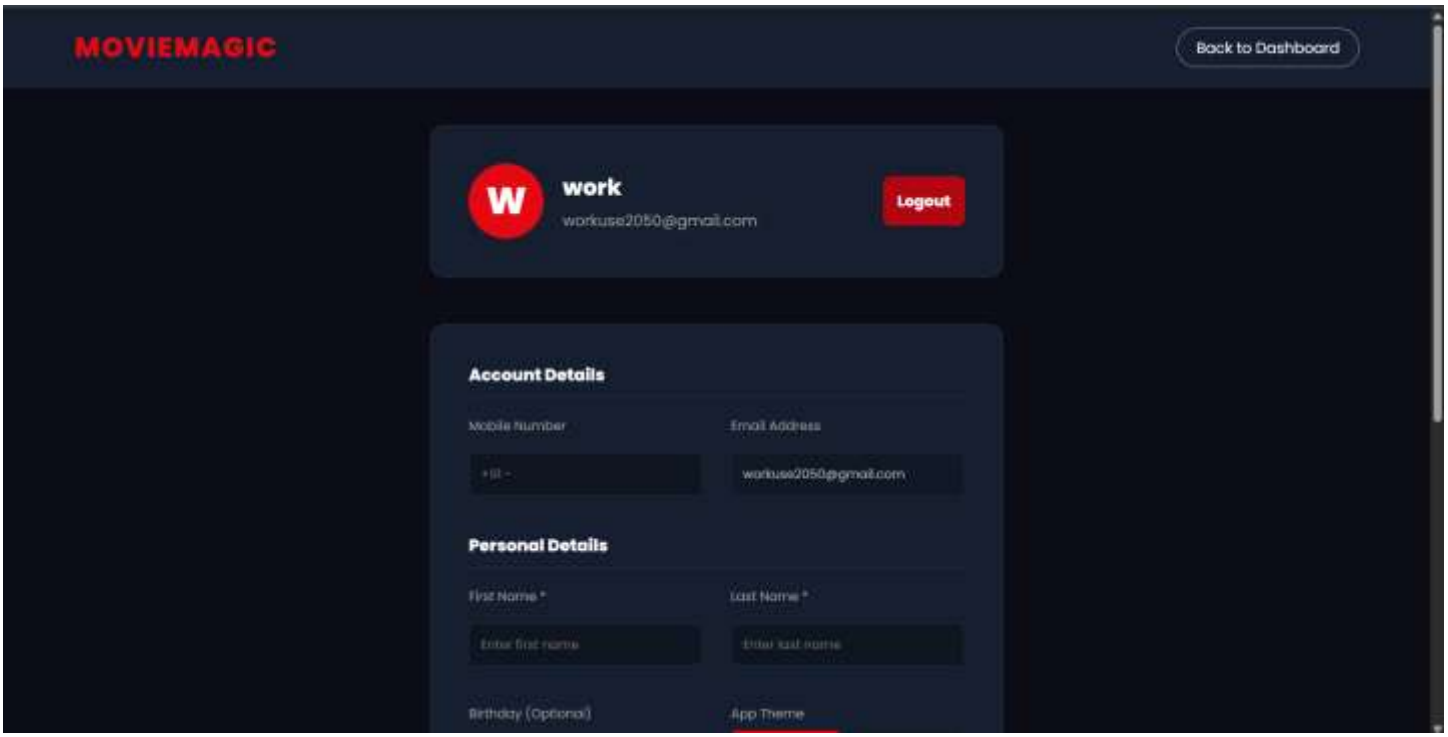

**Register Page:**



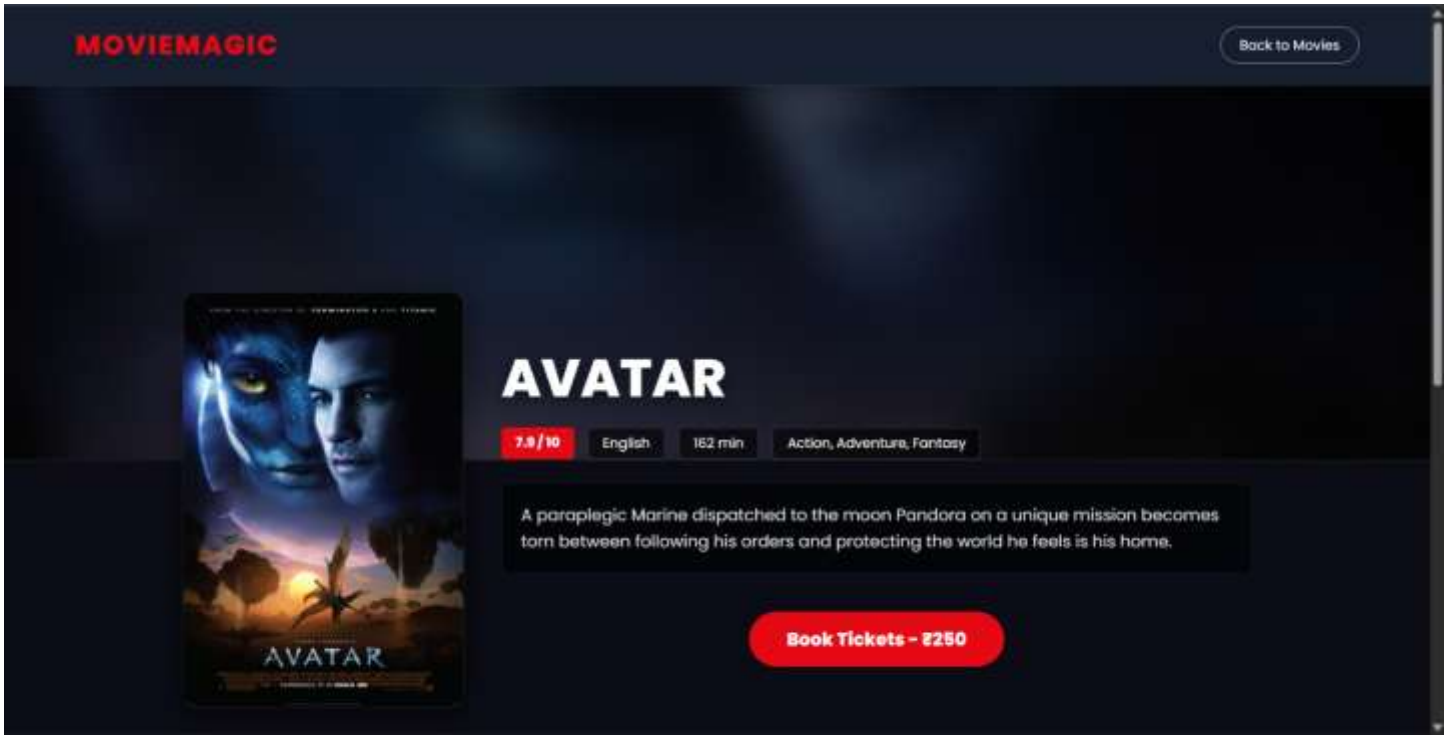
**Dashboard page:**



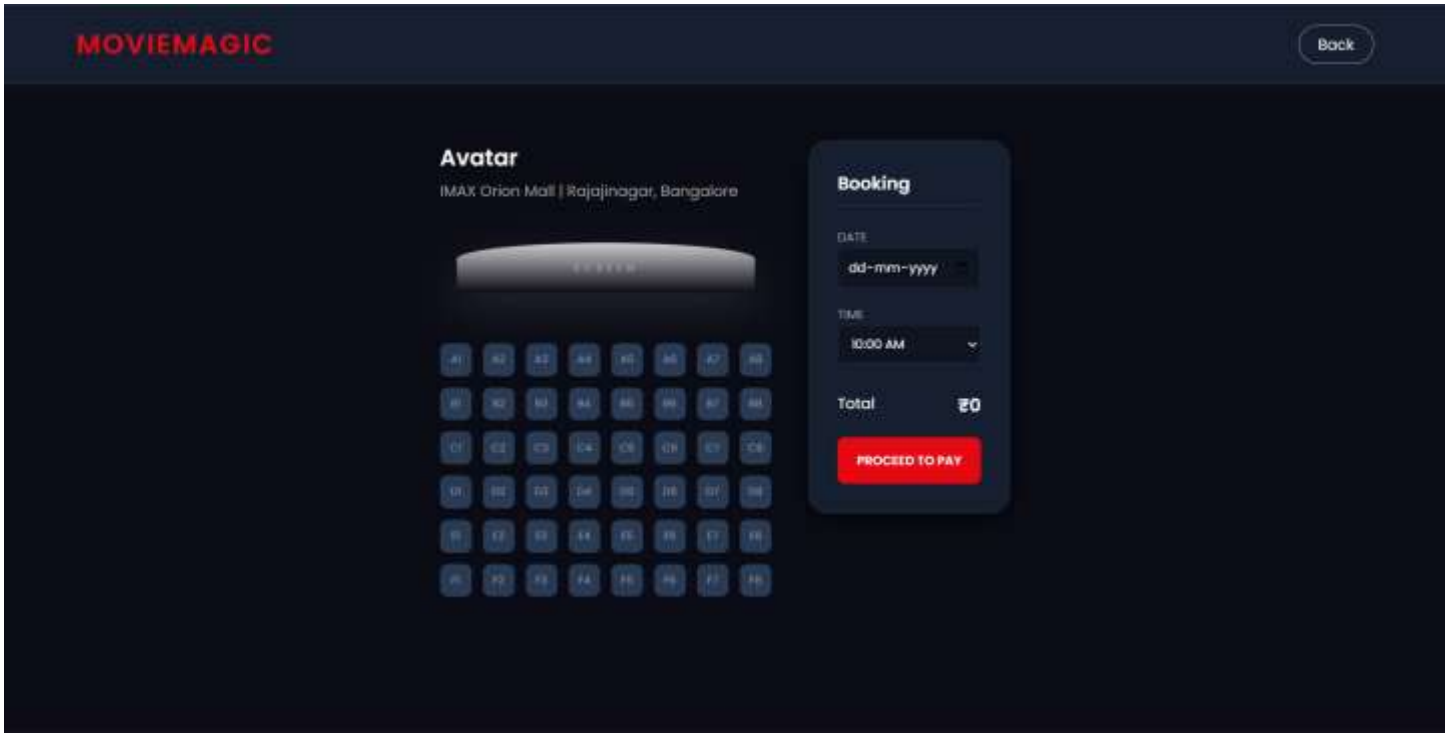

Profile Page:



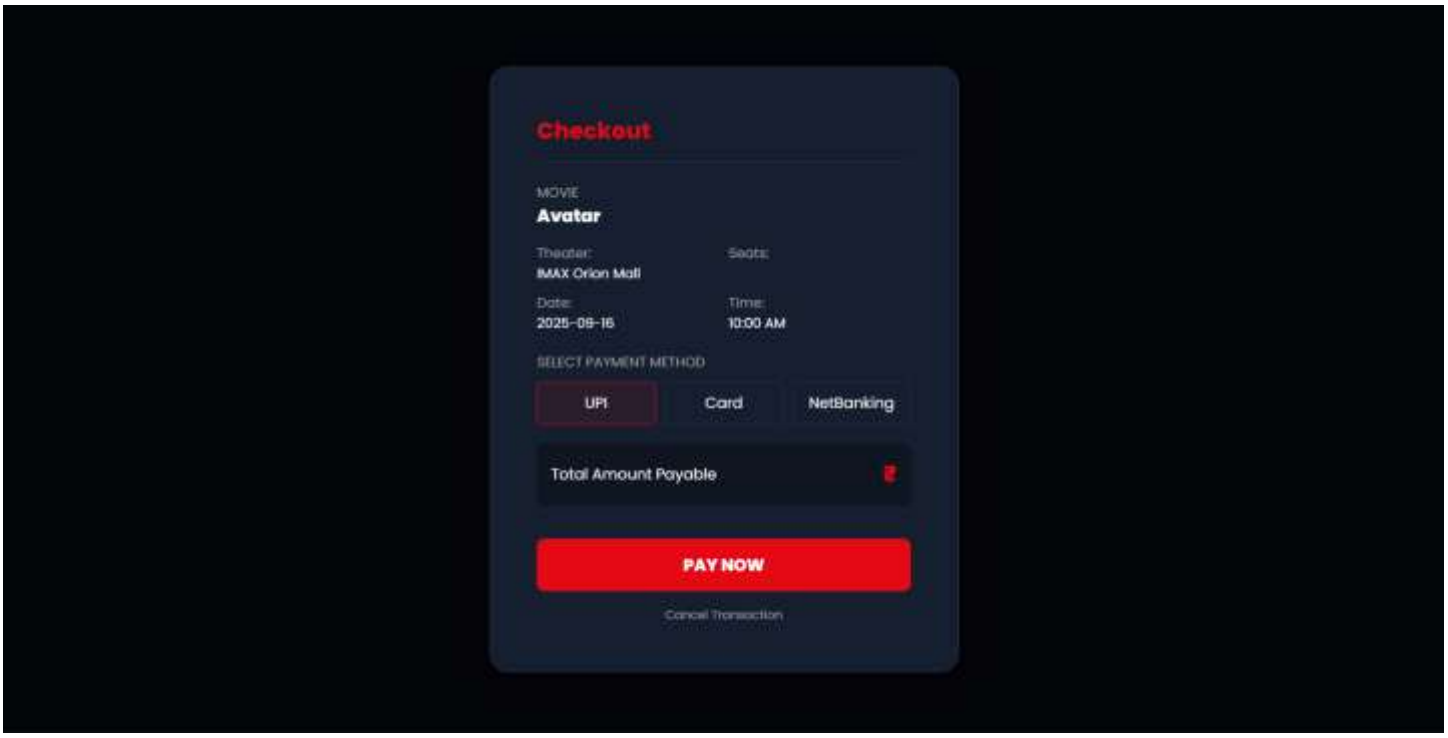
**Movie Page:**



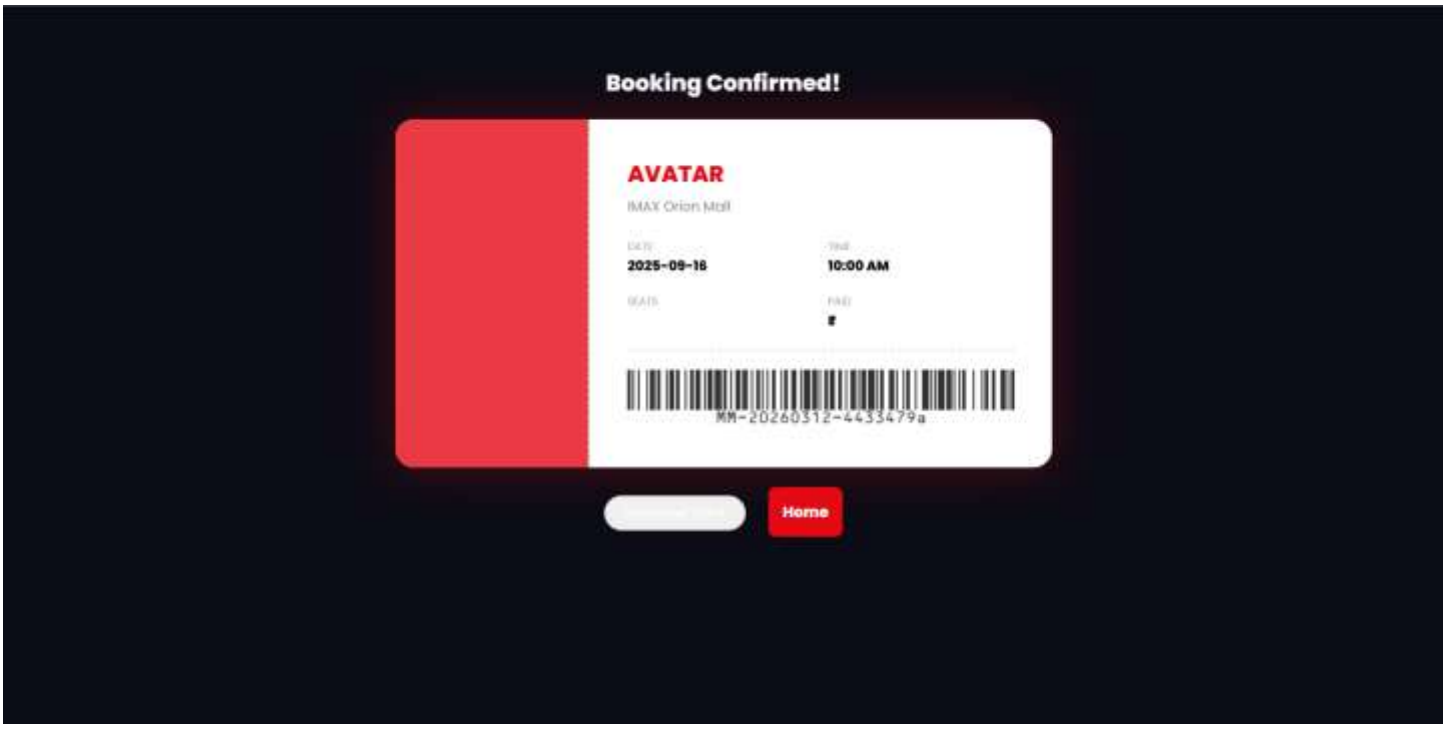

Booking Page:



CheckOut Page:

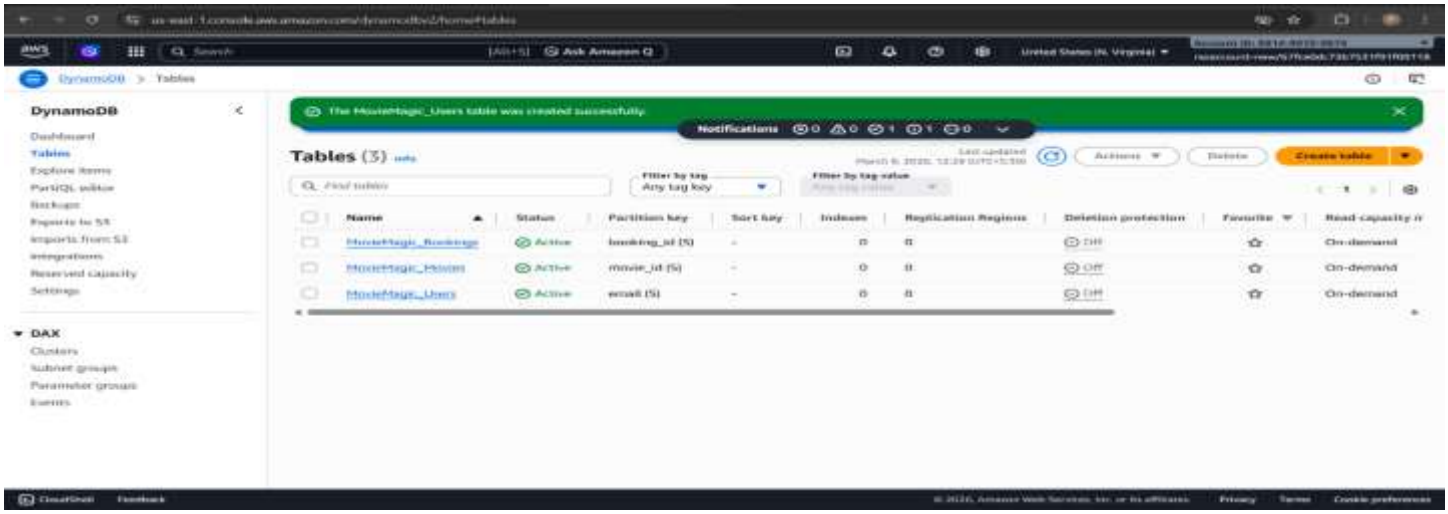



**Bookings Page:**



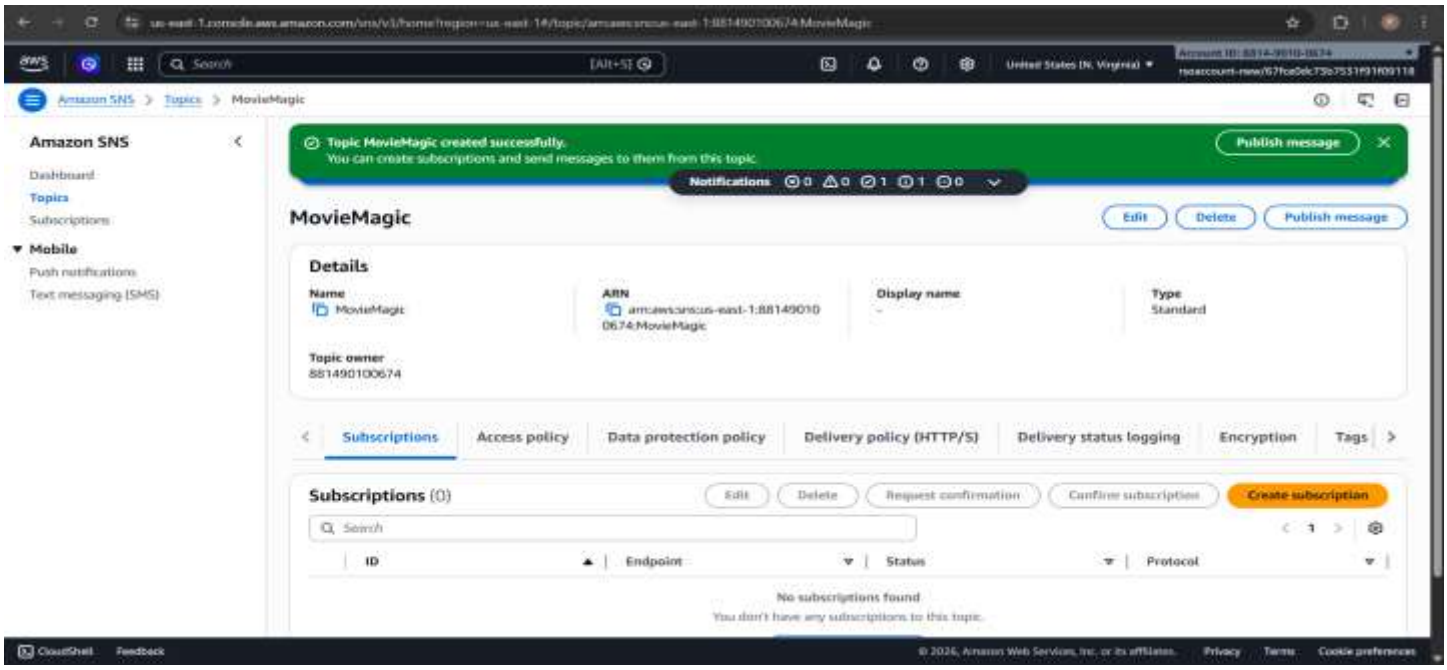
**Dynamodb Database updations :**

1. **Movie-users table :**




**Description:** Store and manage user credentials, login counts, and names in the `movie-Users` table on DynamoDB for authentication and tracking purposes.

2. **SNS table :**



**Description:** Use the Bookings table in DynamoDB to store user movie bookings including date, destination, movie details, and payment information for each transaction.

3. **Mail to the User:**

**AWS Notifications** <no-reply@sns.amazonaws.com>

Tue, Apr 15, 9:11PM (3 days ago)

to me

✅ Booking Confirmed!
SRS Travels | Non-AC Sleeper | 5:30 PM on 2025-04-17 for ₹1220

--

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
https://sns.ap-south-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:ap-south-1:557690616836:BookingConfirmation:da5ad931-7e8f-4ac8-b771-ec2e0cfbaf23&Endpoint=instantlibrary2@gmail.com

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at https://aws.amazon.com/support

## Conclusion:

The **MovieMagic** website has been successfully developed as a modern and user-friendly **online movie ticket booking platform** using a scalable and cloud-ready architecture. Built with a **Flask backend** and a responsive, template-driven frontend, the application enables users to register, log in securely, browse currently available movies, view detailed movie information (including trailers), and proceed through an end-to-end booking flow with seat selection, booking summary, and confirmation.

By leveraging **AWS DynamoDB** for storing user details, movie listings, and booking transactions, MovieMagic supports efficient real-time data management with reliable persistence. Additionally, the integration of **AWS SNS** allows the system to trigger booking confirmation notifications after successful ticket confirmation, improving communication and enhancing the overall user experience. The platform also includes an **Admin Portal**, enabling administrators to add, edit, and delete movie listings, ensuring that the system remains easy to maintain and update.

In summary, **MovieMagic** demonstrates the implementation of a complete ticket booking workflow with secure authentication, dynamic movie management, and cloud-based data handling. It provides a practical solution for simplifying the cinema booking process while highlighting the potential of combining web technologies with AWS services to build responsive and scalable real-world applications.